

C++17 DATA STRUCTURES & ALGORITHMS

数据结构与算法 完整教学与底层原理分析

基于经典教材（CLRS / Sedgewick / Weiss）知识体系
融合 C++17 原生实现、内存演化分析与数据结构不变量证明

语言标准: C++17 | 构建系统: CMake / MinGW64

版本: 2026 最终版 | 核心库: 18 个独立演进模块

数据结构与算法 — 完整教学

目录

第一部分：绪论

- [第 1 章 复杂度分析与 ADT](#)

第二部分：线性结构

- [第 2 章 线性表](#)
- [第 3 章 栈与队列](#)

第三部分：串

- [第 4 章 字符串匹配 \(KMP\)](#)

第四部分：树与二叉树

- [第 5 章 二叉树基础与遍历](#)
- [第 6 章 二叉搜索树](#)
- [第 7 章 堆与优先队列](#)
- [第 8 章 平衡二叉树](#)
- [第 9 章 多路搜索树](#)
- [第 10 章 多维搜索树 \(KD 树\)](#)

第五部分：图

- [第 11 章 图论算法](#)

第六部分：查找

- [第 12 章 哈希表](#)
- [第 13 章 并查集](#)

第七部分：排序

- [第 14 章 排序算法](#)

第八部分：高级字符串

- [第 15 章 后缀树与后缀数组](#)

附录

- [A: 数据结构选择速查](#)
 - [B: 常见面试题与练习](#)
 - [C: 常见陷阱与注意事项](#)
-

第一部分：绪论

本部分定位：建立评价算法好坏的语言，理解数据结构的"接口 vs 实现"分离思想。这是全书所有后续章节的通用基础。

第 1 章 复杂度分析与 ADT

难度：☆☆☆☆☆ | 代码：[complexity_adt.cpp](#)

前置知识：(无前置依赖，本章为全书理论基础。)

学习目标：理解算法效率的评价方法，掌握大 O 表示法的含义与计算规则，理解 ADT 的思想。

1.1 什么是"好"算法？

评价算法的两个维度：

- **时间复杂度：**随着输入规模 n 的增长，运行时间的增长率
- **空间复杂度：**随着 n 的增长，内存占用的增长率

我们关心的是**增长率 (Growth Rate)**—— $n \rightarrow 2n$ 时时间是翻倍还是平方？精确秒数依赖硬件，增长率是算法本身的属性。

1.2 三大渐进符号

符号	名称	含义	类比
O (Big-Oh)	上界	增长 不会比 $g(n)$ 快	上限
Ω (Omega)	下界	增长 至少和 $g(n)$ 一样快	下限
Θ (Theta)	紧确界	恰好和 $g(n)$ 同阶	精确

以插入排序为例：

- 最好 $\Theta(n)$ ：数组已有序，每次只比较 1 次
- 最坏 $\Theta(n^2)$ ：数组完全逆序
- 平均 $\Theta(n^2)$
- $\rightarrow$ 我们说"插入排序的时间复杂度是 $O(n^2)$ "， O 表示最坏情况上界

1.3 常见复杂度阶级

$n = 100$ 时各阶级的量级对比：

```
O(1)      → 1
O(log n)   → ≈7
O(n)       → 100
O(n log n) → ≈700
O(n²)      → 10,000
O(n³)      → 1,000,000
O(2ⁿ)      → 1.27×10³⁰    ← 宇宙年龄级别！
O(n!)      → 9.33×10¹⁵⁷   ← 完全不可行
```

对数复杂度的直观理解：每次操作把问题规模砍半。100 个元素只需 ~7 步，100 万个元素也只需 ~20 步。

$O(n \log n)$ 是什么？ 做 n 次操作，每次操作是对数级别。典型：归并排序——把问题分成两半 ($\log n$ 层)，每层处理所有元素 ($O(n)$)。

复杂度与实际时间对照（假设每秒 10^8 次基本操作）：

n 上限	可行算法复杂度	典型算法
10^8	$O(n)$	线性扫描、计数排序
10^6	$O(n \log n)$	归并排序、快速排序
10^4	$O(n^2)$	插入排序、冒泡排序
500	$O(n^3)$	Floyd 最短路
25	$O(2^n)$	暴力枚举子集
10	$O(n!)$	全排列搜索

这张表是算法竞赛和系统设计中的“第一过滤器”：看到 n 的规模，就知道需要什么级别的算法。

1.4 如何分析代码复杂度？

1. **顺序结构：**取最大。 $O(n^2) + O(n) = O(n^2)$
2. **嵌套循环：**取乘积。外层 $O(n) \times$ 内层 $O(n) = O(n^2)$
3. **if-else：**取最大分支
4. **对数循环：** `for(i=1; i<n; i*=2)` $\rightarrow O(\log n)$
5. **递归：**画递归树或用主定理

1.5 递归复杂度分析 — 主定理 (Master Theorem)

对于递归式 $T(n) = a \cdot T(n/b) + f(n)$ ：

条件	结论
$f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \cdot \log n)$
$f(n) = \Omega(n^{\log_b a + \epsilon})$ 且 $a \cdot f(n/b) \leq c \cdot f(n)$, $c < 1$	$T(n) = \Theta(f(n))$

实例：

```

归并排序:  $T(n) = 2T(n/2) + O(n)$ 
 $a=2, b=2, \log_2 2=1, f(n)=O(n)=\Theta(n^1)$ 
→ 情况 2:  $T(n) = \Theta(n \log n)$ 

二分查找:  $T(n) = T(n/2) + O(1)$ 
 $a=1, b=2, \log_2 1=0, f(n)=O(1)=\Theta(n^0)$ 
→ 情况 2:  $T(n) = \Theta(\log n)$ 

 $T(n) = 4T(n/2) + O(n^2)$ 
 $a=4, b=2, \log_2 4=2, f(n)=O(n^2)=\Theta(n^2)$ 
→ 情况 2:  $T(n) = \Theta(n^2 \log n)$ 

```

1.6 均摊分析的思想

某些操作偶尔很慢，但平均下来每次 $O(1)$ 。**不是求期望，而是保证 m 次连续操作的总时间 $\leq m \times$ 均摊复杂度。**

经典例子——**动态数组扩容**：

```

每次 push 均摊  $O(1)$ :
- 大多数 push 只需  $O(1)$  (直接放末尾)
- 偶尔需要扩容复制  $O(n)$ 
-  $n$  次 push 总代价  $\leq 3n \rightarrow$  均摊  $O(1)$ 

```

记账法 (Accounting Method)：给每次便宜操作“多收一点费”，把余额存起来支付未来昂贵操作的账单。动态数组每次 push 收 3 元（1 元拷贝自己，2 元预付未来扩容）。

二进制计数器：从 0 数到 n ，每次加 1 翻转若干低位比特。翻转总次数 $\leq 2n \rightarrow$ 均摊 $O(1)$ 。这是均摊分析的另一个经典案例。

1.7 抽象数据类型 (ADT)

ADT = 数据 + 操作的**逻辑契约**，与具体实现分离。说白了：**定义“能做什么”，不定义“怎么做”。**

```
栈 ADT = {  
    数据：同类型元素的线性序列  
    操作：  
        push(x): 将 x 放入栈顶  
        pop(): 移除栈顶元素  
        top(): 返回栈顶元素（不移除）  
        isEmpty(): 判断栈是否为空  
}
```

可以实现为数组（顺序栈）或链表（链式栈）
—使用者不需要知道内部实现

ADT 的威力在于**封装**：只要接口不变，底层实现可以任意替换。这就是面向对象编程的根源思想。

ADT 与测试：同一个 ADT 的多个实现（如数组栈 vs 链表栈）应该通过同一套测试用例验证——这正是“接口与实现分离”带来的工程价值。编写面向 ADT 的测试，可以不做任何修改就验证所有实现。

本章小结：

- 复杂度分析是评价算法的通用语言—— O 描述增长率而非绝对速度
- 三大符号各有用途： O 是上界（最常用）， Ω 是下界， Θ 是精确阶
- 主定理覆盖了绝大多数分治递归式的分析
- 均摊分析不等同于“平均”：它是 m 次操作总代价的保证
- ADT 桥接了数学规格说明与工程实现，是现代软件设计的基石
- 核心技能：看到一个循环，能说出它的 O 阶——这在后续每一章都会用到

思考题：

1. 为什么 $O(\log_a n) = O(\log_b n)$ 对任意 $a, b > 1$ 成立？为什么大 O 中可以不写对数的底？
 2. 对递归式 $T(n) = 4T(n/2) + O(n^2)$ 应用主定理。如果 $f(n)$ 改为 $O(n)$ ，结果会如何变化？
 3. 设计一个操作序列，使动态数组在扩容策略（每次 push 的均摊复杂度）为 $O(1)$ 时，单次 push 的最坏代价是 $\Theta(n)$ 。解释均摊分析为什么仍然成立。
-

第二部分：线性结构

本部分学习路线：线性表（顺序表 → 链表）→ 受限线性表（栈 → 队列 → 变体）

核心思想：数据元素之间存在**一对一**的线性关系。线性结构是一切高级数据结构的基本构件——BFS 用队列，DFS 用栈，表达式求值同时使用两者。

线性结构的四个层级：

- 1. 无限制：线性表——任意位置插入删除
- 2. 一端受限：栈——仅栈顶操作
- 3. 两端受限：队列——队尾入队、队首出队
- 4. 混合变体：双端队列、最小栈、双栈模拟队列——组合产生新能力

第 2 章 线性表

难度：☆☆☆☆☆ | 代码：[Linked list.cpp](#)

前置知识：[第 1 章](#) 复杂度分析（理解 $O(1)$ vs $O(n)$ 操作的含义，为选择顺序表还是链表提供判断依据）。

学习目标：理解顺序存储与链式存储的本质区别，掌握单向/双向/循环链表的操作和核心技巧。

2.1 线性表的两种实现

2.1.1 顺序表（数组实现）

```
int data[MAX_SIZE]; // 存储空间
int length;         // 当前元素个数 (≤ MAX_SIZE)
```

操作	复杂度	实现要点
随机访问	$O(1)$	直接下标寻址
尾部插入	$O(1)$	<code>data[length++] = val</code>
中间插入	$O(n)$	插入位置后的所有元素后移一位
删除	$O(n)$	删除位置后的所有元素前移一位
查找	$O(n)$	顺序扫描；若有序可用二分 $O(\log n)$

关键概念：逻辑长度 ≠ 物理容量——length 告诉程序"当前用了多少"，容量告诉程序"还能装多少"。这是贯穿整个课程的基础概念（栈、队列、堆、哈希表都有类似的"满"判断）。

2.1.2 链表 vs 顺序表 对比

	顺序表（数组）	链表
存储方式	连续内存	节点 + 指针
随机访问	O(1)	O(n)
插入/删除	O(n)（需要移动）	O(1)（已有位置引用时）
缓存友好	是（连续内存）	否（内存分散）
空间开销	无额外指针	每节点 1-2 个指针
动态扩容	需要复制	天然支持
适用场景	频繁随机访问、尾部操作	频繁头部/中间增删

选择原则：随机访问多 → 数组；插入删除多 → 链表；两者都频繁 → 平衡树（见第 6-8 章）。

2.2 单向链表

每个节点包含 `data` 和 `next` 指针。

```
head → [10|next] → [20|next] → [30|next] → nullptr
```

操作	复杂度	实现要点
头插	O(1)	<code>newNode->next = head; head = newNode</code>
尾插	O(n)	需遍历到末尾（可维护 tail 指针优化到 O(1)）
删除	O(n)	需找到前驱节点来修正链接
反转	O(n)	三指针法：prev, curr, next

反转算法（三指针法）：

```

prev = nullptr, curr = head
while curr != nullptr:
    next = curr->next    // 暂存下一个节点
    curr->next = prev    // 翻转：当前指向前一个
    prev = curr         // 前移
    curr = next         // 前移
head = prev             // prev 指向原链表的末尾 = 新头

```

示例：1→2→3→null → null←1←2←3

2.3 双向链表

每个节点有 `prev` 和 `next` 两个指针。

```

nullptr ← [prev|10|next] ↔ [prev|20|next] ↔ [prev|30|next] → nullptr

```

优势：删除给定节点只需 $O(1)$ ，因为可以直接通过 `prev` 找到前驱。

```

// 双向链表删除节点 node 只需两行：
node->prev->next = node->next;
node->next->prev = node->prev;
// 对比单向链表：需要从 head 遍历找到 node 的前驱 →  $O(n)$ 

```

2.4 循环链表

尾节点指向头节点，形成环。经典应用：约瑟夫问题—— n 个人围成一圈，每次数到第 m 个人出局。

```

n=5, m=3:
1→2→3→4→5→(回到1)
第1轮数到3 → 3出局
第2轮从4开始数3 → 1出局
... 直到只剩一人

```

2.5 链表的核心技巧

快慢指针 (Floyd's Tortoise and Hare)

```

slow 每次走 1 步，fast 每次走 2 步：
检测环：    slow 和 fast 相遇 → 有环
找环入口：  相遇后 slow 回 head，两者每次走 1 步，再次相遇点 = 环入口
找中点：    fast 到末尾时 slow 恰好在中间

```

为什么 fast 和 slow 一定相遇？ fast 每次比 slow 多走 1 步。如果有环，fast 进入环后会逐步"追上"slow，相当于跑圈套圈。由于差距每次缩小 1，必然相遇。

哨兵节点 (Dummy Node)

在链表头前加一个虚拟节点，统一处理空链表和头节点修改的边界情况。

```
// 合并两个有序链表 — 用哨兵简化代码
ListNode* merge(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->data < b->data) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next; // 哨兵的下一个才是真正头节点
}
```

本章小结：

- 线性表的两种存储方式（顺序 vs 链式）各有适用场景，没有绝对的优劣
- 链表的指针操作是后续树结构的基础——左/右指针本质上就是链表的 next 指针推广
- 三指针反转、快慢指针、哨兵节点是链表操作的三大核心技巧
- 理解“为什么双向链表删除是 $O(1)$ 而单向链表不是”——这揭示了一个深层道理：**结构的冗余（额外指针）换取操作的效率**

思考题：

1. 递归反转单链表：写出递归实现，并追踪 `1→2→3→null` 的调用栈。迭代和递归版本的空间复杂度分别是多少？
2. 证明 Floyd 判环算法中，slow 和 fast 必定在 slow 走完第一圈之前相遇。
3. 设计一个支持以下操作的结构：`getMiddle()` 返回链表中点 ($O(1)$)，`push(x)` 在头部插入 ($O(1)$)，`pop()` 移除头部 ($O(1)$)。

第 3 章 栈与队列

难度：★☆☆☆☆ | 代码：[linear_structures.cpp](#)

前置知识： [第 1 章](#) 均摊分析（双栈模拟队列的均摊 $O(1)$ 分析），[第 2 章](#) 链表插入/删除操作。

学习目标： 掌握栈的 LIFO 特性和队列的 FIFO 特性，理解它们的应用场景，掌握变体结构的实现思路。

3.1 栈 (Stack) — LIFO

后进先出 (Last In First Out)。 就像一摞盘子——后放上去的先拿走。

操作	栈内容	说明
push(1)	[1]	入栈
push(2)	[1,2]	
push(3)	[1,2,3]	
pop()	[1,2]	移除并返回 3
top()	[1,2]	返回 2（不移除）

数组实现（顺序栈）：

- `top_index` 指向栈顶，-1 表示空
- push: `top_index++; data[top_index] = val` → O(1)
- pop: `top_index--` → O(1)

链表实现（链式栈）：

- 链表头部 = 栈顶 → 头插头删均为 O(1)。为什么不用尾部？pop 需要找前驱，O(n)。

栈的经典应用：

应用	描述
括号匹配	左括号入栈，右括号与栈顶匹配
逆波兰表达式	数字入栈，运算符弹出两个数计算
函数调用	递归本质是系统栈帧的压入弹出
撤销操作	Ctrl+Z：操作压栈，撤销时弹出

括号匹配详细追踪：

```

遍历 "{[()]}"：

'{' → push('{')           栈：{
 '[' → push '['           栈：{[
 '(' → push '('           栈：{[(
 ')' → pop, 栈顶='(' 匹配 ✓  栈：{[
 ']' → pop, 栈顶='[' 匹配 ✓  栈：{
 '}' → pop, 栈顶='{' 匹配 ✓  栈：空
遍历结束 + 栈空 → 合法 ✓

反例 "[()]"：
']' → pop, 栈顶='(' ≠ ']' → 不合法 ✗

```

中缀表达式转后缀（调度场算法）：

中缀: "3 + 4 × 2 / (1 - 5)"
后缀: "3 4 2 × 1 5 - / +"

规则:

数字 → 直接输出

运算符 → 栈顶优先级 $\geq$ 当前则弹出, 然后入栈

'(' → 直接入栈

')' → 弹出直到遇到 '('

优先级: $\times/\div > +/ -$

后缀表达式求值追踪 ("3 4 2 × 1 5 - / +"):

```
3 → push          栈: [3]
4 → push          栈: [3, 4]
2 → push          栈: [3, 4, 2]
× → pop 2, pop 4, 4×2=8  栈: [3, 8]
1 → push          栈: [3, 8, 1]
5 → push          栈: [3, 8, 1, 5]
- → pop 5, pop 1, 1-5=-4  栈: [3, 8, -4]
/ → pop -4, pop 8, 8/-4=-2  栈: [3, -2]
+ → pop -2, pop 3, 3+(-2)=1  栈: [1]
结果 = 1 ✓
```

3.2 队列 (Queue) — FIFO

先进先出 (First In First Out)。就像排队——先来先服务。

操作	队列内容	说明
enqueue(1)	[1]	入队
enqueue(2)	[1,2]	
enqueue(3)	[1,2,3]	
dequeue()	[2,3]	移除并返回 1

循环数组实现:

```
front 指向队首, rear 指向队尾下一个空位
判空: front == rear
判满: (rear + 1) % capacity == front (牺牲一个位置)

入队: data[rear] = val; rear = (rear+1) % capacity
出队: val = data[front]; front = (front+1) % capacity
```

链表实现:

- 维护 head (队首) + tail (队尾) 指针
- 入队: tail->next = newNode; tail = newNode $\rightarrow O(1)$
- 出队: head = head->next $\rightarrow O(1)$

3.3 双栈模拟队列详解

入队 $O(1)$ ，出队均摊 $O(1)$ 。

栈 A (入队栈) 栈 B (出队栈)
push → [] [] ← pop

入队 1,2,3: A=[1,2,3], B=[]
出队: B 空 → 将 A 全部倒入 B: B=[3,2,1]
pop B → 1 ✓

再次入队 4: A=[4], B=[3,2]
出队: B 非空 → 直接 pop B → 2 ✓

关键: 每个元素最多"倒"一次 (从 A 到 B), 所以均摊 $O(1)$

为什么这本质上是"两次翻转恢复顺序"?

- 栈的 pop 返回的是插入顺序的逆序
- 把 A 倒入 B 翻转一次, 从 B pop 再翻转一次 → 两次翻转 = 恢复原始 FIFO 顺序
- 数学上: $(\text{push 序列})^R \rightarrow (\text{A 倒入 B})^R \rightarrow \text{pop} = \text{原始 push 序列}$

3.4 单调栈

栈内元素保持单调 (递增或递减), 用于高效解决"下一个更大/更小元素"类问题。

问题 1: 找出数组中每个元素右边第一个比它大的元素
输入: [2, 1, 2, 4, 3]
输出: [4, 2, 4, -1, -1]

算法 (单调递减栈):
for x in arr:
 while 栈非空 and x > 栈顶:
 栈顶找到了"右边第一个更大" = x
 弹出并记录
 x 入栈 (保持栈内递减)

$O(n)$: 每个元素最多入栈一次、出栈一次

问题 2: 柱状图中最大矩形 (Largest Rectangle in Histogram):

```
heights = [2, 1, 5, 6, 2, 3]
```

单调递增栈，遇到下降的柱子时计算矩形面积：

```
i=0: 2入栈          栈: [2(index=0)]
i=1: 1<2 → 弹出2, maxArea=2×1=2  栈: [1(0)]
i=2: 5入栈          栈: [1(0),5(2)]
i=3: 6入栈          栈: [1(0),5(2),6(3)]
i=4: 2<6 → 弹出6, maxArea=6×1=6
      2<5 → 弹出5, maxArea=5×2=10
      2>1 → 2入栈      栈: [1(0),2(4)]
i=5: 3>2 → 3入栈      栈: [1(0),2(4),3(5)]
最后依次弹出计算...    最大面积=10 ✓
```

直观理解：弹出柱子时，说明当前高度"结束了一段"—计算以该高度为高的最大矩形

3.5 变体结构总览

变体	描述	核心实现
双端队列 (Deque)	两端都可入队出队	双向链表
最小栈	$O(1)$ 获取最小值	主栈 + 辅助栈（同步存最小值）
双栈模拟队列	入 $O(1)$ ，出均摊 $O(1)$	A管入队，B管出队
循环队列	空间复用	取模运算判满判空
优先队列	按优先级出队	堆（见第 7 章）

问题类型 → 推荐变体：

问题特征	推荐结构
需要回溯	栈（DFS）
按顺序处理	队列（BFS）
两端操作	双端队列（滑动窗口）
需要快速获取最值	最小/大栈
模拟排队 + 查找	优先队列（堆）

本章小结：

- 栈和队列的核心在于"受限"——限制操作端不是为了限制能力，而是为了获得**确定性**（LIFO / FIFO）
- 栈的 LIFO 保证让"最近的历史"最先被访问（撤销、回溯、括号匹配），队列的 FIFO 保证公平和顺序
- 双栈模拟队列揭示了栈作为"序列翻转器"的本质——两次翻转恢复原序

- 单调栈展示了"利用有序性剪枝 $O(n^2) \rightarrow O(n)$ "的优化范式，这一思想在图的最短路和 KD 树剪枝中会反复出现
- 变体结构（最小栈、Deque）的本质是**在基础结构上叠加额外信息**，这种"扩展"模式在后面章节会不断见到

思考题：

1. 给定入栈序列 $1, 2, 3, \dots, n$ ，允许在任何时刻 pop。哪些输出排列是不可能的？描述不可能的排列的特征（提示：考虑逆序对和 312 模式）。
 2. 双栈模拟队列中，单个 dequeue 的最坏时间复杂度是多少？设计一个操作序列使其达到最坏。为什么均摊分析仍然成立？
 3. 设计一个支持 $O(1)$ `getMin()` 的队列（不是栈！）。提示：需要不同于最小栈的策略。
-

第三部分：串

本部分定位：字符串匹配是“预处理后加速查询”这一思想的首次登场。第 4 章的 KMP 是全书第一个需要静下心来理解“为什么能优化”的算法——这种“利用已知信息避免重复工作”的思维模式将贯穿后续所有高效算法（BST 的有序遍历、哈希的 $O(1)$ 查询、后缀结构的预建索引）。

第 4 章 字符串匹配（KMP）

难度：★★★☆☆ | 代码：[string_match.cpp](#)

前置知识：[第 1 章](#) 均摊分析（while 循环中 j 只减不增，减小次数 $\leq$ 增加次数，故均摊 $O(1)$ ），[第 2 章](#) 线性表基本概念。

学习目标：理解暴力匹配的缺陷，掌握 KMP 前缀函数和 next 数组的含义，能手动计算 next 数组并追踪匹配过程。

4.1 问题描述

给定文本串 T （长度 n ）和模式串 P （长度 m ），找出 P 在 T 中所有出现的位置。

4.2 暴力匹配 $\rightarrow O(n \cdot m)$

```
for i = 0 to n-m:
    for j = 0 to m-1:
        if T[i+j] != P[j]: break
    if j == m: 匹配成功
```

最坏： $T = \text{"AAAAAAAAAB"} , P = \text{"AAAAB"} \rightarrow O(n \cdot m)$

最坏情况可视化（每次匹配到最后一个字符才失败）：

```
T = "AAAAAAAAAB", P = "AAB"
i=0: A A A|B → 比较了 3 次才失败
i=1:  A A B| → 比较了 3 次，其中前 2 次是重复劳动
...
总比较次数  $\approx (8-3+1) \times 3 = 18$ ，而  $n=8, m=3, n \times m=24 \rightarrow$  接近  $O(n \cdot m)$ 
```

每次失配时 T 回退到 $i+1$ 重头开始——已经比较过的字符信息被完全丢弃。

4.3 KMP 的核心洞察

失配时模式串已经匹配了 j 个字符 $\rightarrow$ 我们知道 T

$$i \dots i + j - 1$$

= P

$$0 \dots j - 1$$

。

关键问题：P

$$0 \dots j - 1$$

的最长相等前后缀长度是多少？

如果这个长度是 k ，那么 P 的前 k 个字符和 T 的后 k 个字符已经自动对齐，**j 直接跳到 k 继续比较，T 无需回退。**

一句话总结：利用已匹配部分的信息，将模式串“滑”到下一个可能匹配的位置。

4.4 前缀函数 π (next 数组)

定义： π

$$i$$

= P

$$0 \dots i$$

的最长相等真前后缀长度（真 = 长度 $< i+1$ ）。

P = "ABABCABAB" 逐位计算 π ：

```
i=0: "A"      → 无真前后缀 →  $\pi[0]=0$ 
i=1: "AB"     → "A"≠"B"    →  $\pi[1]=0$ 
i=2: "ABA"    → "A"=="A"    →  $\pi[2]=1$ 
i=3: "ABAB"   → "AB"=="AB"   →  $\pi[3]=2$ 
i=4: "ABABC"  → 无           →  $\pi[4]=0$ 
i=5: "ABABCA" → "A"=="A"    →  $\pi[5]=1$ 
i=6: "ABABCAB" → "AB"=="AB"  →  $\pi[6]=2$ 
i=7: "ABABCABA" → "ABA"=="ABA" →  $\pi[7]=3$ 
i=8: "ABABCABAB" → "ABAB"=="ABAB" →  $\pi[8]=4$ 
```

$\pi = [0, 0, 1, 2, 0, 1, 2, 3, 4]$

4.5 计算前缀函数 $O(m)$

```
computePrefix(P):
     $\pi[0] = 0$ 
     $j = 0$  //  $j$  = 当前匹配的前缀长度
    for  $i = 1$  to  $m-1$ :
        while  $j > 0$  and  $P[i] \neq P[j]$ :
             $j = \pi[j-1]$  // 退回较短的可能匹配长度
        if  $P[i] == P[j]$ :
             $j++$  // 匹配扩展
         $\pi[i] = j$  // 记录最长前后缀长度
```

循环不变式：在每轮 for 循环开始时， $j = P[0..i-1]$ 的最长相等前后缀长度。

while 为什么是均摊 $O(1)$? j 每次最多 +1 (for 循环每轮)，减少的总次数不可能超过增加的总次数。所以 m 轮总操作 $\leq 2m \rightarrow$ 均摊每轮 $O(1)$ 。

4.6 KMP 匹配 $O(n)$

```
KMP(T, P):
     $j = 0$ 
    for  $i = 0$  to  $n-1$ :
        while  $j > 0$  and  $T[i] \neq P[j]$ :
             $j = \pi[j-1]$  //  $T$  不后退！只调整  $P$  的指针
        if  $T[i] == P[j]$ :
             $j++$ 
        if  $j == m$ :
            匹配成功！位置 =  $i - m + 1$ 
             $j = \pi[j-1]$  // 继续找下一个匹配
```

4.7 KMP 匹配完整追踪

```
T = "ABABDABACDABABCABAB" (n=19)
P = "ABABCABAB" (m=9)
 $\pi = [0, 0, 1, 2, 0, 1, 2, 3, 4]$ 

i=0..3:  $T[0..3]="ABAB"$ ,  $P[0..3]="ABAB" \rightarrow j=4$ 
i=4:  $T[4]=D$ ,  $P[4]=C \rightarrow$  失配！
     $j=\pi[3]=2 \rightarrow P$  跳到位置 2
i=4:  $T[4]=D$ ,  $P[2]=A \rightarrow$  失配！
     $j=\pi[1]=0 \rightarrow P$  跳到位置 0
i=4:  $T[4]=D$ ,  $P[0]=A \rightarrow$  失配！  $\rightarrow i++$ 

i=5..9: 逐步匹配  $\rightarrow j=3$ 
i=10:  $T[10]=C$ ,  $P[3]=B \rightarrow$  失配！  $j=\pi[2]=1$ 
i=10:  $T[10]=C$ ,  $P[1]=B \rightarrow$  失配！  $j=\pi[0]=0$ 
...

i=10..18: 重新匹配，最终  $j=9=m \rightarrow$  匹配成功！位置 =  $18-9+1 = 10 \checkmark$ 

★ 注意  $T$  的指针  $i$  自始至终只前进，从不后退！
```

4.8 第二个追踪实例（展示多次跳转）

```
P = "AAACAAA", n = 7
π = [0, 1, 2, 0, 1, 2, 3]

T = "AAACAAAAC"
匹配:
i=0..2: T="AAA", P="AAA" → j=3
i=3: T[3]=C, P[3]=A → 失配! j=π[2]=2
i=3: T[3]=C, P[2]=A → 失配! j=π[1]=1
i=3: T[3]=C, P[1]=A → 失配! j=π[0]=0 ← 连续回退三次
i=3: T[3]=C, P[0]=A → 失配!
i=4..7: "AAAA" 逐步匹配 → j=4 (但 P 只有 7 个字符)
...
最终匹配成功 @ i=3..9 "CAAAAAC"
```

★ 此例展示 while 循环可以连续回退多次——但总的回退次数不超前进次数

4.9 nextval 优化

当 `P[j] == P[π[j]]` 时，回退到 π

j

仍会失配（字符相同），可直接跳更远：

```
nextval[j] = (P[j] == P[π[j]]) ? nextval[π[j]] : π[j]
```

4.10 延伸：KMP 与其他单模式匹配算法

- **Boyer-Moore**：从右向左匹配，坏字符+好后缀规则，实践中最快， $O(n/m)$ 最好情况
- **Sunday**：比 BM 更简洁，检查 T 中模式窗之后的下一个字符
- **KMP**：被选入教材的原因——概念最清晰，保证 $O(n+m)$ ，且前缀函数的思想推广到了 AC 自动机和后缀数组

本章小结：

- KMP 的核心贡献不是算法本身，而是一个思想：“部分匹配的信息可以被结构化地编码和复用”
- 前缀函数 π 将这种信息编码为数组，使匹配问题从“回溯搜索”变为“DFA 式的状态转移”
- $O(m)$ 的前缀计算使用同样的思想递归地应用在模式本身上（模式匹配自己），使得算法设计高度自治
- 均摊分析在 KMP 中有两处应用：前缀计算的 while 循环，和匹配阶段的 while 循环——j 的增加次数是其减少次数的上界

思考题：

1. 手动计算 `P = "ABACABAB"` 的 π 数组，并用算法伪代码追踪验证。
2. 证明 KMP 匹配阶段指针 i 从不回退。为什么这意味着 KMP 对在线/流式输入友好？

3. 如果要在文本 T 中找出与 P 最长公共子串的起始位置，如何修改 KMP？（提示：考虑在 π 数组中额外存储“以 i 结尾的最长匹配长度”）
-

第四部分：树与二叉树

本部分学习路线：二叉树与遍历 → BST 基本操作 → 堆（另一种特殊的完全二叉树）→ 平衡树（AVL → 红黑树 → 伸展树）→ 多路树（B → B+）→ 多维树（KD）

核心思想：数据元素之间存在**一对多**的层次关系。树是递归定义的——每个子树也是一棵树。

螺旋上升的学习框架：

1. **Ch5 建立通用概念：**树的术语和遍历——此时树只是一个"形状"，不附带搜索语义
2. **Ch6 引入搜索语义：**BST 给树加上大小比较的约束。但暴露了退化问题——"只加约束、不强制平衡"是不够的
3. **Ch7 换个视角：**堆也是二叉树，但完全不用于搜索——它维护的是优先级关系。完全二叉树 + 数组存储是全新的范式。 $O(\log n)$ 且无退化——引出"树形约束 = 深度保证"的道理
4. **Ch8 解决退化：**三种平衡策略（严格 / 宽松 / 自适应），各自有不同的工程取舍
5. **Ch9 跳出二叉树：**多路树用"一个节点装多个 key"换取更低的树高——本质是空间换 I/O
6. **Ch10 跳出单维度：**KD 树把"比较"推广到多维空间，同时引出几何剪枝这一全新的优化策略

学完本部分，你应该能从多个维度理解"树到底是什么"——递归容器、搜索结构、优先级维护器、空间划分器。

第 5 章 二叉树基础与遍历

难度：★★☆☆☆ | 代码：[binary_tree.cpp](#) 遍历部分

前置知识：[第 2 章](#) 链表（链式二叉树的左/右指针是链表 next 指针的自然推广——从"一个后续"变"两个后续"）。

学习目标：掌握二叉树的定义、存储方式（链式/数组），熟练四种遍历的手工与代码实现。

5.1 树的术语

A

/

|

\

B

C

D

/

\

/

\

E

F

G

H

/

\

I

J

← 根 (Root)
 ← 分支节点
 ← 叶节点 (Leaf)

节点的度：子节点个数 (A度=3, B度=2, E度=2)
 树的度：所有节点度的最大值
 深度/层次：从根到节点的边数 (I的深度=3)
 高度：从节点到最深叶子的边数 (B的高度=2)

树 vs 二叉树：

- 树：每个节点可以有任意多个子节点
- 二叉树：每个节点最多 2 个子节点，分为左子和右子（左右不可交换——即使只有一边，左和右也代表不同的结构）

5.2 二叉树的性质

以下三个性质是二叉树理论的经典定理，也是常考点：

性质 1：二叉树的第 i 层最多有 $2^{(i-1)}$ 个节点（根所在层 $i=1$ ）。

性质 2：深度为 k 的二叉树最多有 $(2^k - 1)$ 个节点。

性质 3（叶节点与二度节点关系）：对于任何非空二叉树， $n_0 = n_2 + 1$ 。

证明：设总边数为 E。每个节点（除根外）由一条边连接父节点 $\rightarrow E = n - 1$ 。另一方面，每个度为 2 的节点贡献 2 条边，度为 1 的贡献 1 条 $\rightarrow E = 2n_2 + n_1$ 。两式相等： $n - 1 = 2n_2 + n_1$ ，代入 $n = n_0 + n_1 + n_2$ ： $n_0 + n_1 + n_2 - 1 = 2n_2 + n_1 \rightarrow$ 消去 n_1 ： $n_0 = n_2 + 1 \checkmark$

三个易混淆术语：

术语	定义	数组存储
满二叉树 (Full)	每个节点有 0 或 2 个子节点	不一定适合
完全二叉树 (Complete)	除最后一层外每层满，最后一层左对齐	✓ 适合
完美二叉树 (Perfect)	每层都满，共 2^k-1 个节点	✓ 适合

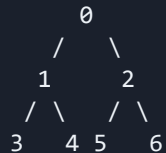
5.3 二叉树的存储

链式存储（最常用）：

```
struct TreeNode {
    int data;
    TreeNode *left, *right;
};
```

数组存储（完全二叉树专用）：

索引 i （从 0 始）：
 父： $(i-1)/2$ 左子： $2i+1$ 右子： $2i+2$



5.4 四种遍历

三种深度优先遍历的本质区别只有一处：**处理当前节点的时机**。

```
// 前序：先处理根
void preorder(Node* t) { if(!t) return; cout<<t->data; preorder(t->left); preorder(t->right);

// 中序：从左子树返回后处理根
void inorder(Node* t) { if(!t) return; inorder(t->left); cout<<t->data; inorder(t->right);

// 后序：处理完左右子树再处理根
void postorder(Node* t){ if(!t) return; postorder(t->left);postorder(t->right);cout<<t->data;
```

三种递归代码**结构完全相同**——唯一的区别是 `cout` 的位置。这体现了遍历的统一框架。

遍历	顺序	时机	典型应用
前序 NLR	根→左→右	进入节点时处理	复制/序列化树
中序 LNR	左→根→右	从左子树返回后处理	BST 有序输出
后序 LRN	左→右→根	离开节点前处理	释放内存（先删子节点）
层序 BFS	逐层从左到右	用队列	按层打印、最短路径



前序: 4 2 1 3 6 5 7 (先访问根)
 中序: 1 2 3 4 5 6 7 (BST: 严格升序!)
 后序: 1 3 2 5 7 6 4 (先处理子节点)
 层序: 4 → 2 6 → 1 3 5 7

层序遍历的"拍快照"技巧:

每层开始前记录 `levelSize = q.size()`，然后只处理恰好这么多节点。不拍快照的话，处理中入队的新节点会导致层边界混乱。

5.5 遍历的递归与迭代

递归实现简洁但可能爆栈。迭代实现需显式使用栈:

中序迭代:

1. 沿左子树一路 `push` 入栈
2. 栈顶弹出并访问
3. 转向其右子树，重复步骤 1

本章小结:

- 二叉树的四种遍历是全部树算法的基础——每个树操作都可以归约为"在某种遍历中加入处理逻辑"
- 三种 DFS 遍历的区别仅在于"处理当前节点"的时机，这决定了它们是自顶向下（前序）还是自底向上（后序）
- 性质 $n_0 = n_2 + 1$ 的证明方法（数边两次）是图论中常见的证明技巧
- 数组存完全二叉树的公式 `父(i-1)/2`，`左2i+1`，`右2i+2` 是第 7 章堆和第 14 章堆排序的基础

思考题:

1. 已知二叉树的前序序列和中序序列，能否唯一确定这棵树？如果是前序+后序呢？给出反例或证明。
2. 写出后序遍历的非递归实现（用单栈）。为什么后序迭代比前序复杂很多？
3. 对于非空二叉树，证明 $n_0 = n_2 + 1$ （叶节点数 = 二度节点数 + 1）。推广到任意树（不限于二叉树）， n_0 与各度节点数之间的关系是什么？

第 6 章 二叉搜索树

难度: ★★☆☆☆ | 代码: [binary_tree.cpp](#) BST 部分

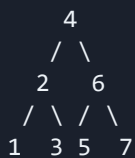
前置知识：[第 5 章](#) 二叉树遍历（特别关心中序遍历 = 升序输出这一性质），[第 1 章](#) 递归分析（BST 操作均为 $O(h)$ ）。

学习目标：掌握 BST 的定义和性质，理解插入/删除/查找的实现逻辑，特别是删除的三种情况和后继替换原理。

6.1 BST 的定义

对于任意节点，左子树所有节点的值 < 当前节点的值 < 右子树所有节点的值。

推论：中序遍历 BST = 严格升序输出。这是 BST 最核心的性质。



中序：1 2 3 4 5 6 7 ← 必然升序！

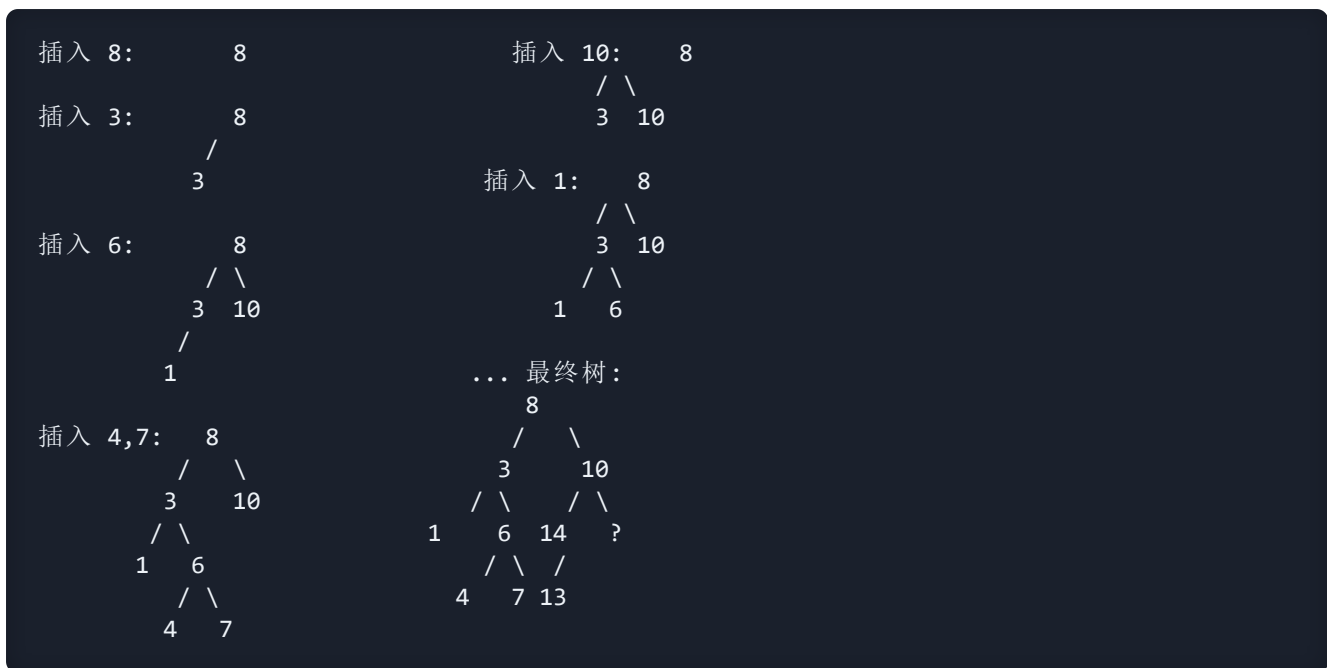
6.2 BST 的基本操作

查找：从根开始，比当前节点小走左边，大走右边。 $O(h)$ 。

插入：从根开始找位置，到达空位就创建新节点。

⚠ **必须在递归返回时赋值给子指针：** `node->Left = BSTinsert(node->Left, val)` 如果只写 `BSTinsert(node->Left, val)` 不接收返回值，新节点不会真正连到树上。

BST 插入分步追踪（依次插入 `[8, 3, 10, 1, 6, 14, 4, 7, 13]`）：



前驱与后继:

- **后继** = 右子树的最小节点（往右一步，然后一直向左走到头）
 - 情况 A: 节点有右子 → 右子树中的最小值
 - 情况 B: 节点无右子 → 沿父链向上，直到某个祖先的左子在该路径上（即"该节点是某个祖先的左子树的节点"）
- **前驱** = 左子树的最大节点（往左一步，然后一直向右走到头）

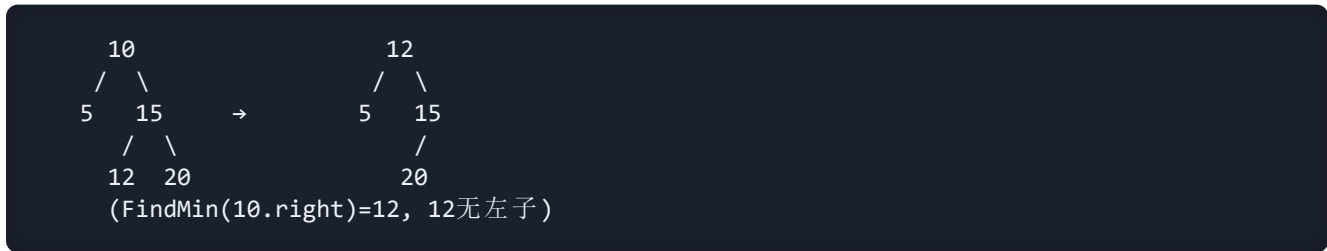
6.3 BST 删除 — 三种情况

情况 1（叶节点）：直接 `delete`，返回 `nullptr`

情况 2（一个子节点）：用子节点替代自己

情况 3（两个子节点）：找后继，用它覆盖值，递归删除后继
后继最多一个右子 → 退化为情况 1/2

删除 10 的完整图解:



为什么用后继替换? 后继 = 比当前节点大的最小节点。它 > 所有左子树节点 ✓, < 所有右子树其他节点 ✓, 替换后 BST 性质不变。

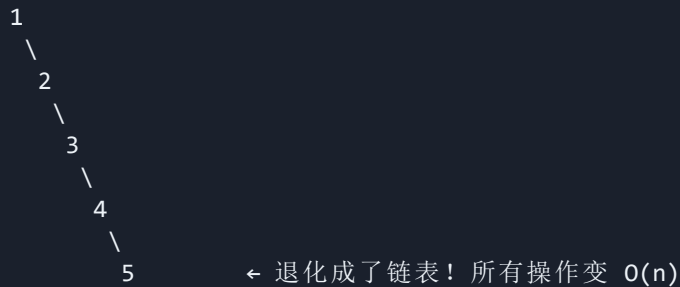
删除经典陷阱:

```
// 错: delete 后父指针仍指向已释放的内存 → 野指针!
BSTdelete(node->Left, val);

// 对: 接收返回值, 父指针被正确更新
node->Left = BSTdelete(node->Left, val);
```

6.4 BST 的退化问题

依次插入 1,2,3,4,5:



BST vs 有序数组: 这是一个重要的工程权衡——

- 有序数组: 二分查找 $O(\log n)$, 但插入/删除 $O(n)$ (需要移动元素)
- 平衡 BST: 查找/插入/删除均为 $O(\log n)$
- BST 通过牺牲查找的常数因子 ($\log_2(n)$ vs $\log_2(n)$, 但是指针跳转), 换取了高效的动态更新

解决方案: 自平衡 BST (AVL、红黑树等) —— 后续章节将详细介绍。

6.5 验证 BST 的合法性

中序遍历 BST 必须是严格升序。用 `int*& prev` 跟踪上一个访问的节点值, 如果当前值 $\leq$ prev $\rightarrow$ 违反 BST 性质。

为什么用 `int*&` 而非 `int&`? `nullptr` 携带"这是第一个节点"的信息——第一个节点没有前驱, 不进行比较。

6.6 增强 BST — $O(\log n)$ 查第 k 小

每个节点额外存储 `leftSize` (左子树的节点总数)。

```
currentRank = leftSize + 1

k == currentRank  $\rightarrow$  命中!
k < currentRank  $\rightarrow$  去左子树 (k 不变)
k > currentRank  $\rightarrow$  去右子树 (k = k - currentRank)
```

这就是 C++ `__gnu_pbds::tree` (有序集合) 的底层原理。

本章小结:

- BST 是最简单的有序映射数据结构—— $O(h)$ 查找/插入/删除, $O(1)$ 最小/最大值
- BST 的核心脆弱点: **h 可能退化为 n**——这是第 8 章的动机
- BST 删除的三个情况 + 后继替换, 是 AVL 和红黑树删除的基础 (它们使用相同的删除骨架, 仅添加重新平衡步骤)
- "接收递归返回值"模式是树操作最常见的错误点——忘记赋值导致物理链接断裂
- BST 的中序有序性使得它天然适合范围查询, 这正是 B+ 树 (第 9 章) 保留的核心特性

思考题:

1. 给定一棵 BST, 将其转换为"Greater Sum Tree": 每个节点的值替换为所有大于它的节点值之和。提示: 反向中序 (右→根→左)。
2. 在 BST 中找两个节点的最近公共祖先 (LCA), $O(h)$ 时间。BST 的性质如何使 LCA 比普通二叉树更简单?
3. 证明: 如果 n 个元素以随机顺序插入空 BST, 期望高度为 $O(\log n)$ 。如果以排序顺序插入会怎样?

第 7 章 堆与优先队列

难度: ★★☆☆☆ | 代码: [heap.cpp](#)

前置知识: [第 5 章](#) 完全二叉树的数组存储方式, [第 6 章](#) BST 的树形结构和退化问题 (堆通过完全二叉树的约束彻底避免了高退化)。

学习目标: 理解堆的完全二叉树性质与数组存储方式, 掌握 push/pop 的上浮/下沉操作, 理解 Top-K 问题的巧解。

7.1 什么是堆?

堆是一棵**完全二叉树**, 满足**堆序性质**:

- **最小堆**: 每个节点值 $\leq$ 子节点值 $\rightarrow$ 根是最小值
- **最大堆**: 每个节点值 $\geq$ 子节点值 $\rightarrow$ 根是最大值

堆 vs BST 对比:

维度	最小堆	BST
核心操作	getMin $O(1)$	search $O(h)$
有序遍历	✗ 不支持	✓ 中序升序
存储方式	数组	指针
退化风险	无 (完全二叉树)	有 (需平衡)
合并复杂度	$O(n)$ (数组堆)	$O(n \log n)$ 重建

堆和 BST 针对完全不同的问题——堆是高效的优先级管理器，BST 是高效的查找表。两者不可互换。

7.2 上浮与下沉

上浮 (UP): 新元素放在末尾，如果比父节点小，就向上交换。

```
push(2) 到 [1, 3, 5, 8, 7]:  
放入末尾 → [1, 3, 5, 8, 7, 2]  
2 < 父(5) → 交换 → [1, 3, 2, 8, 7, 5]  
2 < 父(3) → 交换 → [1, 2, 3, 8, 7, 5]  
2 > 父(1) → 停止 ✓
```

下沉 (DOWN): 弹出堆顶后用末尾元素填补，如果比子节点大，就和较小子节点交换。

```
pop() 从 [1, 3, 5, 8, 7]:  
取出堆顶 1，末尾 7 放到堆顶 → [7, 3, 5, 8]  
7 > 左子 3 (3 是较小者) → 交换 → [3, 7, 5, 8]  
7 > 左子 8? 7 < 8 → 停止 ✓
```

为什么选"较小"子节点? 换上去的值必须 $\leq$ 两个子节点。选较小的换上去，保证 $\leq$ 另一个没动的子节点。

7.3 建堆为什么是 $O(n)$

从最后一个非叶节点开始，自底向上逐个堆化。每个节点下沉的深度取决于它在树中的高度——底层大部分节点下沉 0-1 层，只有顶层少数节点下沉深。

```
精确推导:  
设  $h = \lfloor \log_2 n \rfloor$ ，第  $i$  层有最多  $2^i$  个节点，每个最多下沉  $h-i$  层  
总下沉次数  $\leq \sum_{i=0}^h 2^i \times (h-i) = O(n)$   
  
用替代法:  $\sum 2^i \times (h-i) = 2^{h+1} - h - 2 \leq 2n$   
→  $O(n)$ ，不是  $O(n \log n)$ !
```

建堆的实际代价远小于直觉预测的 $O(n \log n)$ ——这是堆结构的一个重要理论性质。

7.4 堆排序完整过程

```
原始: [4, 10, 3, 5, 1]  
建最大堆: [10, 5, 3, 4, 1]  
  
第1轮: 10↔1 → [1, 5, 3, 4|10] (10就位)  
第2轮: 5↔4 → [4, 1, 3|5, 10] (5就位)  
第3轮: 4↔3 → [3, 1|4, 5, 10] (4就位)  
第4轮: 3↔1 → [1|3, 4, 5, 10] (完成, 升序)
```

7.5 Top-K 问题 — 最小堆做门槛过滤器

从海量数据中找最大的 k 个元素， $O(n \log k)$ vs 全排序 $O(n \log n)$ 。

```
维护大小为 k 的最小堆：
堆未满 → 直接入堆
堆已满：
- val > 堆顶（堆顶 = 当前 Top-K 中最小的）→ 踢掉堆顶，val 入堆
- val ≤ 堆顶 → 跳过

最后堆中就是前 k 大的元素
```

为什么用最小堆存"最大"元素？ 堆顶是"入门门槛"——比门槛小的元素不可能进入 Top-K。如果用最大堆，堆顶是最大值，无法判断何时淘汰旧元素。

变体：数据流中位数（双堆法）：

```
维护两个堆：
maxHeap（较小半）：允许快速查最大值
minHeap（较大半）：允许快速查最小值
平衡：|maxHeap| - |minHeap| ≤ 1

中位数 = maxHeap.top()（奇数个时）或 (maxHeap.top()+minHeap.top())/2（偶数个时）
每次插入  $O(\log n)$ 
```

7.6 左式堆 — 可合并的优先队列

难度：★★★☆☆ | 代码：[heap.cpp](#) Part 2

普通二叉堆合并需要 $O(n)$ 。左式堆通过维护**左倾性质** ($npl(\text{左}) \geq npl(\text{右})$)，支持 $O(\log n)$ 合并。

NPL（零路径长）：到最近"没有两个子节点"的节点的最短距离。空节点 $npl = -1$ ，叶节点 $npl = 0$ 。

合并算法核心：

```
merge(n1, n2):
    if n1 > n2: swap(n1, n2)           // 确保 n1 根最小
    n1->right = merge(n1->right, n2)    // 沿右路径递归
    if npl(左) < npl(右): swap(左右)   // 维护左倾性质
    n1->npl = npl(右) + 1               // 右子树决定 npl
```

为什么沿右路径？ 左倾性质保证右路径最短 → 右路径长度 = $O(\log n)$ 。递归沿右路径进行，保证 $O(\log n)$ 时间复杂度。如果沿左路径合并，可能退化为 $O(n)$ 。

插入 = $\text{merge}(\text{root}, \text{新节点})$ ，删除 = $\text{merge}(\text{left}, \text{right})$ → 都是 $O(\log n)$ 。

左式堆 vs 斜堆：斜堆不存储 npl ，每次回溯无条件交换左右，均摊 $O(\log n)$ ，实现更简单。

本章小结：

- 堆是全课程最"紧凑"的数据结构——一个数组 + 两个辅助函数（UP/DOWN）实现全部优先队列操作
- 堆的核心设计智慧：**用结构约束（完全二叉树 + 堆序）换取强保证（ $O(\log n)$ 操作 + 无退化风险）**
- 左式堆展示了另一种设计哲学：放松"完全二叉树"约束，改用 npl 不变量，从而获得 $O(\log n)$ 合并能力
- Top-K 用最小堆的方案是"不直觉但高效"的经典案例——理解了它就是理解了堆的灵活运用
- 双堆中位数是 Top-K 思想的自然延伸，也是流式算法设计的经典范例

思考题：

- 实现 d-ary heap（每个节点有 d 个子节点）。d=3 时，父节点和子节点的索引公式是什么？d-ary heap 在什么场景下优于二叉堆？
- 给定一个无限的数据流，维护中位数，每次插入 $O(\log n)$ 。写出完整的插入逻辑（处理两个堆的大小平衡）。
- 斐波那契堆可以实现 $O(1)$ 均摊 decrease-key。基于二叉堆的结构，直觉上为什么二叉堆做不到这一点？

第 8 章 平衡二叉树

本章路线：AVL（严格平衡，1962）→ 红黑树（工业首选，1972）→ 伸展树（自适应，1985）

前置知识：[第 6 章 BST](#) 的插入/删除/旋转操作，[第 1 章](#) 均摊分析（伸展树的均摊 $O(\log n)$ 分析）。

8.1 AVL 树

难度：★★★★☆ | 代码：[avl tree.cpp](#)

学习目标：理解平衡因子的概念，掌握四种旋转的触发条件和执行方式。

平衡因子 (BF) = 左子树高度 - 右子树高度。合法范围 $\{-1, 0, 1\}$ 。

平衡因子更新决策表（插入路径回溯时）：

子节点在父节点的哪边	父 BF 变化	新 BF 含义
插入左子树	++	0→停止（子树高度不变），1→继续向上，2→旋转修复
插入右子树	-	0→停止，-1→继续向上，-2→旋转修复

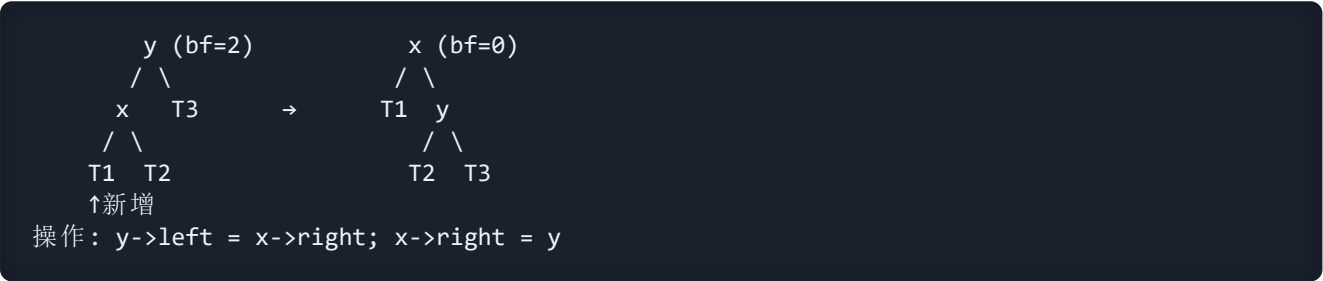
关键洞察：旋转后，子树高度恢复到插入前的值 → 祖先的平衡因子不再受影响 → 插入最多 2 次旋转（LR/RL 需要两次）。这是 AVL 插入比删除高效得多的原因。

四种失衡与旋转：

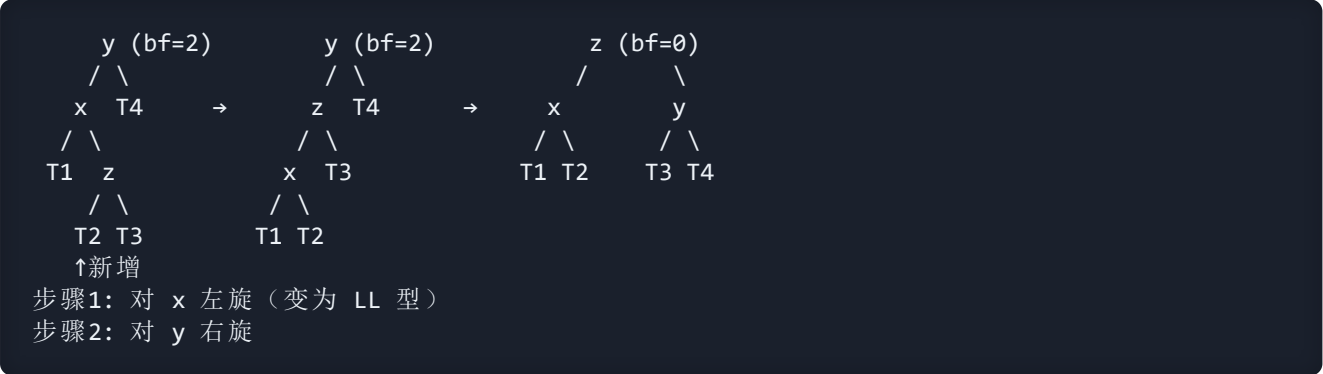
类型	条件	旋转方式
LL	左子树的左边插入	右单旋
RR	右子树的右边插入	左单旋
LR	左子树的右边插入	左旋左子 → 右旋根
RL	右子树的左边插入	右旋右子 → 左旋根

口诀：LL 右旋，RR 左旋，LR 左右旋，RL 右左旋。

LL 型旋转图解：



LR 型旋转图解：



插入 vs 删除：

- 插入：最多旋转 2 次就能恢复平衡
- 删除：可能需要 $O(\log n)$ 次旋转（删除旋转后子树高度可能减 1，祖先可能再度失衡）

8.2 红黑树

难度：★★★★★ | 代码：[redblacktree.cpp](#)

学习目标：理解五条性质如何保证平衡，掌握插入修复的三种情况，理解工程中为何选择红黑树。

C++ `std::map`、`std::set`，Java `TreeMap`、`TreeSet`，Linux CFS 调度器，`epoll` 事件管理——全部基于红黑树。

五条性质：

- 1. 节点是红色或黑色
- 2. 根是黑色
- 3. NIL/叶节点是黑色
- 4. 红色节点的两个子节点都是黑色（不能连续红）
- 5. 从任意节点到叶子的每条路径黑节点数相同（黑高相等）

为什么这些性质能平衡？

性质 4 + 性质 5 → 最长路径 ≤ 2 × 最短路径。

- 最短路径：全黑，长度 = bh（黑高）
- 最长路径：红黑交替，红 ≤ 黑 → 长度 ≤ 2·bh
- → 树高 ≤ 2·log(n+1) → O(log n)

深层不变量：黑高才是真正的不变量——所有路径的黑节点数量相同。红色节点是"借来的高度"——它们拉长了路径但不增加黑高。正因为此，红色节点不能连续出现（不允许连续两次"借高度"）。理解了这一点，五条性质就是自然推论。

红黑树 vs AVL 树：

	AVL 树	红黑树
平衡条件	严格 (BF∈{-1,0,1})	宽松（五条性质）
树高	≤ 1.44 log n	≤ 2 log n
查找	略快（更矮）	略慢
插入删除	旋转多	最多 3 次旋转
适用	查多写少	读写均衡

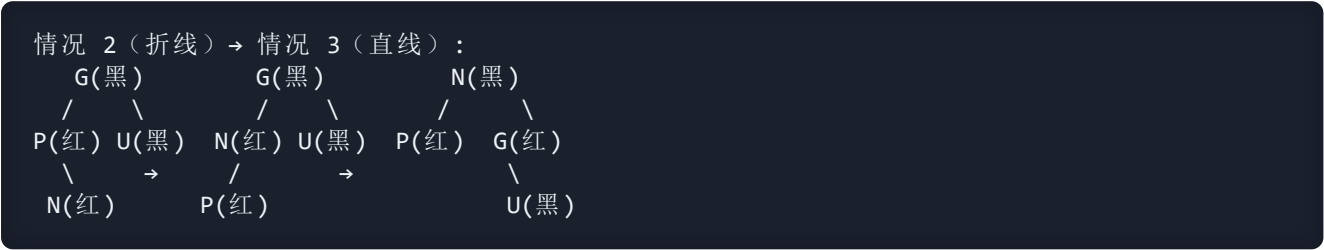
插入修复（三步走）：

新节点默认是**红色**（不破坏性质 5，只可能破坏性质 2/4）。

约定：N=新节点，P=父，U=叔叔，G=祖父（P 为 G 左子为例）

情况	叔叔颜色	N 位置	处理
1	红	任意	P/U 变黑，G 变红，问题上移
2	黑	右子（折线）	对 P 左旋 → 变情况 3
3	黑	左子（直线）	P 变黑，G 变红，对 G 右旋 → 完成

修复示例：



口诀：叔红变色往上走，叔黑同侧一旋定，叔黑异侧先转同。

删除简介：删除黑色节点时黑高减少，需通过兄弟节点"借"或"合并"修复。

情况	条件	处理
1	兄弟 S 是红色	P/S 变色 + 旋转 → 换成黑兄弟
2	S 和侄子全黑	S 变红，问题上移
3	S 黑，远侄黑，近侄红	S 变红，近侄变黑，旋转 → 变情况 4
4	S 黑，远侄红	S 取 P 色，P/远侄变黑，旋转 → 完成

8.3 伸展树

难度：★★★★☆ | 代码：[splaytree.cpp](#)

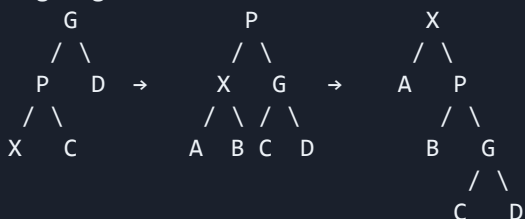
学习目标：理解"访问即平衡"的自适应思想，掌握 Zig/Zig-Zig/Zig-Zag 三种旋转。

核心哲学：每次访问后把目标节点旋转到根——最近访问的节点最容易被再次访问（类 LRU）。

三种 Splay 操作：

情况	条件	操作
Zig	P 是根	单旋 X
Zig-Zig	X 和 P 同侧	先旋 P（绕 G），再旋 X
Zig-Zag	X 和 P 异侧	旋 X 两次

Zig-Zig (同侧, 先父后子):

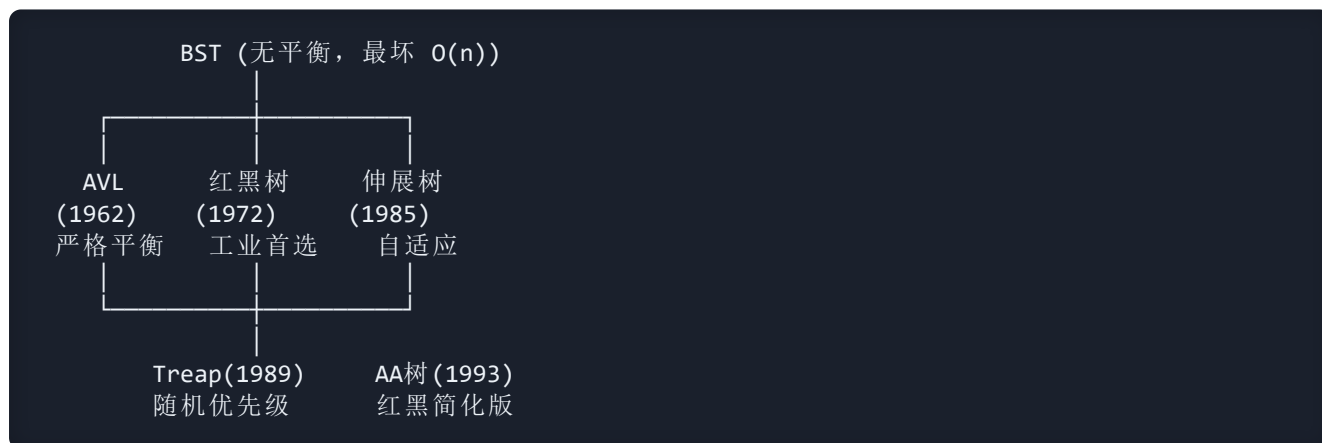


为什么 Zig-Zig 必须先旋转 P (父节点) 再旋转 X? 如果先旋 X 再旋 X (两次"子先上"), P 仍在 X 下方, G 仍在 P 下方 → 结果不是二叉搜索树形式。先旋父、后旋子的"双层旋转"是保证均摊 $O(\log n)$ 的数学关键——Tarjan 的 Access Lemma 证明依赖于此。

三棵树对比:

	伸展树	AVL	红黑树
额外存储	无	高度(int)	颜色(bool)
单次最坏	$O(n)$	$O(\log n)$	$O(\log n)$
均摊	$O(\log n)$	$O(\log n)$	$O(\log n)$
局部性	自动优化	无	无

8.4 平衡树族谱



本章小结:

- 三种策略, 一个目标: 保持树浅。AVL 靠严格强制 (即时再平衡), 红黑树靠宽松约束 (用颜色间接保证), 伸展树靠完全重组 (访问即平衡)
- 选择取决于负载特征: 读多 → AVL, 写多 → 红黑树, 时间局部性强 → 伸展树
- 三者共享同一根: BST + 旋转作为基本重组操作。理解 BST、旋转、平衡条件之间的关系, 就理解了树的世界观
- 红黑树的"黑高不变量"是解码其设计的钥匙——五条性质只是满足这个不变量的语法糖

思考题：

1. 依次插入 [10, 20, 30, 40, 50, 25] 到空 AVL 树，逐步画出每次插入和旋转后的树。
2. 证明：在红黑树中，最长路径 $\leq 2 \times$ 最短路径。为什么这个界限足够保证 $O(\log n)$ 深度？
3. 伸展树的最坏查找是 $O(n)$ ，而 AVL 保证 $O(\log n)$ 。为什么在某些场景下（例如缓存、搜索引擎）仍然选择伸展树？

第 9 章 多路搜索树

本章路线：B 树（磁盘优化） $\rightarrow$ B+ 树（数据库索引王者）

前置知识：第 6 章 BST（理解 BST 的搜索语义），第 8 章 红黑树（理解平衡条件如何保证 $O(\log n)$ 深度）。建议同时了解磁盘 I/O 的基本概念。

9.1 B 树

难度：★★★★☆ | 代码：[multiwaytree.cpp](#)

学习目标：理解“减少磁盘 I/O”的设计动机，掌握 B 树的分裂与合并操作。

为什么需要 B 树？

内存 > 磁盘约 10^5 倍。BST 每次读 1 个 key $\rightarrow$ I/O 太多。B 树一个节点存几十~几百个 key $\rightarrow$ 树极矮 $\rightarrow$ I/O 极低。

10 亿条记录：

BST: $h \approx \log_2(10^9) \approx 30$ 层 $\rightarrow$ 最坏 30 次磁盘 I/O

B 树: $h \approx \log_{250}(10^9) \approx 3$ 层 $\rightarrow$ 最多 3-4 次磁盘 I/O

$\rightarrow$ 快 10 倍！

m 阶 B 树性质：

1. 每个节点最多 m-1 个 key，最多 m 个子节点
2. 非根节点至少 $\lceil m/2 \rceil - 1$ 个 key
3. 根节点至少 1 个 key
4. 所有叶节点在同一层（完美平衡）

3 阶 B 树 = 2-3 树（每个节点 1-2 个 key）。

分裂过程（3 阶为例）：

节点 $[10, 20]$ + 插入 $15 \rightarrow [10, 15, 20]$ 溢出
 → 中间 key 15 提升到父节点
 → $[10]$ 和 $[20]$ 成为两个子节点
 → 父节点可能也溢出 → 递归向上分裂 → 可能树高 $+1$

删除与下溢处理:

从 2-3 树中删除 key:
 1. key 在叶节点 → 直接删除
 2. 删除后叶节点为空 → 下溢!
 → 优先向兄弟"借"一个 key (兄弟有富余时)
 borrow: $[5]$ 和 $[10, 20] \rightarrow 5$ 上到父, 10 下到左 → $[10]$ 和 $[5, 20]$
 → 兄弟也穷 → 合并: $[5]$ 和 $[10] +$ 父 key $15 \rightarrow [5, 10, 15]$
 合并后父节点可能也下溢 → 递归向上

9.2 B+ 树

难度: ★★★★★ | 代码: [bplustree.cpp](#)

学习目标: 理解 B+ 树与 B 树的本质区别, 掌握数据库选 B+ 树的四大理由。

B+ 树 vs B 树:

	B 树	B+ 树
数据位置	内部节点也存	仅叶节点存
内部节点函数	key 是数据	key 是路由副本
叶节点连接	独立	双向链表相连
查找终点	可能在内部	必须到叶节点
范围查询	中序遍历	链表扫描 $O(k)$

结构示意:

```

    内部节点:  [10]      [30]      ← 仅做路由
                /  \    /  \
              [1,5] [15,20] [35,40] ...
    叶节点+链表: [1,v]↔[5,v] [15,v]↔[20,v] ... ← 存数据+链表
    
```

四大优势:

- 范围查询高效:** `SELECT ... WHERE id BETWEEN 100 AND 200`, 找到 100 后顺链表扫描即可
- I/O 更友好:** 内部节点不存数据 → 更"瘦" → 一个磁盘页装更多索引 → 树更矮
- 查询性能稳定:** 任何查找都到叶节点, 时间恒定

4. 聚集索引：InnoDB 叶节点直接存行数据

叶节点分裂的关键：分裂时右半的最小 key 提升到父节点，但**保留在右叶中**（副本而非移走）——因为内部节点只做路由，数据必须在叶节点中完整保留。

B+ 树叶分裂示例：

LeafBefore: [10, 20, 30, 40] ← 4 个 key（假设 $m=5$ ，最多 4 个 key）

插入 25 → 分裂：

Parent 获得路由器：25（同时保留在右叶）

LeftLeaf: [10, 20]

RightLeaf: [25, 30, 40] ← 25 既在叶中又在父中！

9.3 2-3 树与红黑树的等价关系

（来自 [multiwaytree.cpp](#) 代码注释，第 47-64 行）

一个 4-node（3 个 key，4 个子节点）可以编码为一棵小的红黑树：

```
2-3-4 树的 4-node: [a, b, c]    → Red-Black 树:  b(黑)
      / | | \                      / \
      / | | \                      a(红) c(红)

3-node: [a, b]                  → Red-Black 树:  b(黑)    或    a(黑)
      / | \                      / \          / \
      / | \                      a(红)        b(红)
```

这解释了为什么红黑树恰好需要 $O(\log n)$ 深度——它本质上是一棵 2-3-4 树，而 2-3-4 树的所有叶节点在同一层。这个对应关系是红黑树复杂度分析的核心工具。

本章小结：

- 多路搜索树是从学术到工业的桥梁——B 树通过扇出降低树高来解决 I/O 瓶颈
- B+ 树在 B 树的基础上加了叶节点链表，这个“小事”让范围查询从 $O(k \log n)$ 变为 $O(k)$ ，彻底改变了数据库查询性能
- 核心设计原则：使节点大小匹配磁盘页大小，每次 I/O 均摊到许多 key 上
- 2-3 树 ↔ 红黑树的等价关系是一个优美的理论连接——红黑树本质上是 2-3-4 树的二叉编码

思考题：

1. 一棵 200 阶的 B 树存储 1000 万条记录。最小和最大高度分别是多少？最坏情况多少次磁盘 I/O？
2. 在 B+ 树中，内部节点的 key 是路由副本。如果从叶节点删除一个恰好也出现在内部节点的 key，应该怎么做？为什么不能仅删除叶节点中的副本？
3. 对比 B+ 树的范围查询和 BST 中序遍历：`SELECT * WHERE key BETWEEN 100 AND 200`。两种方案各需要多少次 I/O？

第 10 章 多维搜索树 (KD 树)

难度：★★★★☆ | 代码：[kdtree.cpp](#)

前置知识：[第 5 章](#) 二叉树遍历，[第 6 章 BST](#) (KD 树是 BST 到 k 维空间的推广)，[第 1 章](#) 均摊分析 (最近邻搜索的均摊 $O(\log n)$)。

学习目标：理解将 BST 推广到多维空间的思路，掌握维度轮换策略和最近邻搜索的剪枝原理。

核心思想：每层按一个维度划分空间，深度 d 按第 $(d \% k)$ 维划分。取该维度中位数作为分割点。

2D 空间划分示意：

第一层 (x 分裂)： $x \leq 5 \mid x > 5$

第二层 (y 分裂)： $y \leq 3 \mid y > 3$ $y \leq 4 \mid y > 4$

KD 树构建追踪 (2D 点集 $[(3,6), (17,15), (13,15), (6,12), (9,1), (2,7), (10,19)]$)：

第一层 (维度 $0=x$, 中位数 ≈ 9):

Left: $[(3,6), (6,12), (2,7)]$ Right: $[(17,15), (13,15), (10,19)]$

第二层 (维度 $1=y$):

Left 分支: y 中位 $\approx 7 \rightarrow (3,6)/(2,7)$ 和 $(6,12)$

Right 分支: y 中位 $\approx 15 \rightarrow (17,15)/(13,15)$ 和 $(10,19)$

最近邻搜索的剪枝：

1. 沿树向下走到底，记录最近距离 `bestDist`
2. 回溯：
 - 更新 `bestDist`
 - 计算 Q 到"另一侧分割平面"的垂直距离
 - 如果垂直距离 $< \text{bestDist} \rightarrow$ 搜索另一侧
 - 否则 $\rightarrow$ 整个另一侧剪枝！

为什么能剪枝？点与 Q 的距离 $\geq$ 到分割平面的垂直距离。
如果垂距已 $> \text{bestDist}$ ，另一侧绝不可能有更近的点。

范围搜索 / KNN：同样的剪枝逻辑适用——如果搜索区域不与子树的包围盒相交，直接跳过整棵子树。

实际应用：

- 空间数据库 (PostGIS)
- 游戏物理引擎 (碰撞检测)
- 计算机视觉 (特征匹配、SIFT)
- 地理信息系统 (GIS) 中的"查找附近餐馆"

平均 $O(\log n)$ ，最坏 $O(n)$ 。实践中剪枝效果显著。但需注意**维度灾难**——当维度 k 增大时，剪枝效率指数级下降，KD 树的实际性能趋近于 $O(n)$ 。

本章小结：

- KD 树将 BST 的 1D 比较推广到 k D 空间——每层轮换维度，等于在多维空间中画"超平面刀片"
- 剪枝策略依赖一个巧妙的几何事实：点到超平面的垂直距离是到另一侧所有点距离的下界
- 维度灾难是 KD 树的实际限制—— $k > 10$ 时几乎无法有效剪枝，需要用 LSH、Ball Tree 等高维专用结构
- KD 树代表了一种更广泛的模式：**空间数据结构用几何约束换取搜索效率**，在平均情况下表现优秀

思考题：

1. 在 2D KD 树中，一个点能同时是分割平面两侧的最近邻候选吗？解释。
 2. 随着维度 k 增加，KD 树的剪枝效率如何变化？什么是"维度灾难"在 KD 树语境中的具体含义？
 3. 另一种策略：不是轮换维度，而是在每个节点选择方差最大的维度进行分割。这种策略的优缺点是什么？
-

第五部分：图

本部分定位：从线性（一对一）到树（一对多），现在进入**图（多对多）**。图的复杂性跃升是本质性的——不是学更多操作，而是学新的**问题分类方式**：遍历、连通性、最短路、生成树。每种问题要求不同的算法范式。图的算法也是对全书数据结构的综合运用：BFS 用队列，DFS 用栈/递归，Dijkstra 用优先队列，Kruskal 用并查集。

第 11 章 图论算法

难度：★★★★★ | 代码：[graph.cpp](#)

前置知识：[第 3 章](#) 队列（BFS） / 栈（DFS），[第 7 章](#) 堆/优先队列（Dijkstra 用堆优化），[第 13 章](#) 并查集（Kruskal 用并查集判环）。

学习目标：掌握图的存储（邻接矩阵/邻接表），理解遍历/拓扑排序/MST/最短路径四大类算法。

11.1 图的基本概念

图 $G = (V, E)$ ：V=顶点集，E=边集。

分类维度	类型	说明
方向	有向/无向	边有方向 vs 边对称
权重	有权/无权	边带有数值 vs 所有等价
连通	连通/非连通	任意两点可达 vs 存在孤岛

11.2 图的存储

邻接矩阵：

```
0 1 2 3
0 [0 1 1 0]
1 [1 0 0 1]
2 [1 0 0 1]
3 [0 1 1 0]
```

空间： $O(V^2)$
判邻接： $O(1)$

邻接表：

```
0 → [1, 2]
1 → [0, 3]
2 → [0, 3]
3 → [1, 2]
```

空间： $O(V+E)$
判邻接： $O(deg)$

选择原则：稠密图($E \approx V^2$) → 矩阵；稀疏图($E \approx V$) → 邻接表。

11.3 图的遍历

	DFS（深度优先）	BFS（广度优先）
数据结构	栈（递归）	队列
路径	不保证最短	保证最短（无权图）
内存	$O(h)$	$O(w)$
适合	找所有路径、拓扑排序	最短路径、层次遍历

DFS：走到底 → 退回 → 换路 BFS：一层一层扩展

DFS 边分类（有向图）——这是图论中 DFS 遍历的核心内容：

DFS 遍历有向图时，每条边正好被分类为四种之一：

- 树边 (Tree Edge)：DFS 树中的边
- 回边 (Back Edge)：指向 DFS 树中祖先的边 → 检测到回边 = 图中有环
- 前向边 (Forward Edge)：指向非直接后代的边
- 横叉边 (Cross Edge)：指向其他分支已访问节点的边

无向图中只有"树边"和"回边"两类（前向边和横叉边不会出现）。

11.4 拓扑排序

问题：DAG 中排成线性序，使每条边 $u \rightarrow v$ 中 u 在 v 前。

Kahn 算法 (BFS + 入度)：

- 1. 计算所有顶点入度
- 2. 入度=0 者入队
- 3. while 队不空：
 - 出队 $v \rightarrow$ 加入结果
 - 对 v 的每个邻接 w ：
 - w 入度 -- $\rightarrow$ 若变 0 则入队
- 4. 结果长度 $< V \rightarrow$ 有环！

思路：入度 0 = 无前置依赖 = 可以安全排在最前面。去掉后，依赖它的节点可能入度变 0。

11.5 最小生成树 (MST)

问题：连通无向带权图中，找一棵边权和最小的生成树。

	Prim (加点法)	Kruskal (加边法)
策略	每次选最短的横切边	按权排序, 不形成环就加
适合	稠密图	稀疏图
数据结构	优先队列	并查集
复杂度	$O(V^2)$ 或 $O(E \log V)$	$O(E \log E)$

Kruskal 为什么用并查集?

每条边 (u,v) : $\text{find}(u) \neq \text{find}(v) \rightarrow$ 不形成环 $\rightarrow$ 加入并 union
 如果 find 相同 $\rightarrow$ 已在同一连通分量 $\rightarrow$ 加这条边会形成环 $\rightarrow$ 跳过

11.6 最短路径

Dijkstra (单源, 边权 ≥ 0):

```
dist[src]=0, 其他= $\infty$ 
重复 V 次:
  u = 未访问中 dist 最小者  $\rightarrow$  标记已访问
  for u 的邻接 v:
    dist[v] = min(dist[v], dist[u] + w(u,v))
```

为什么不能有负权边? 每个节点只"处理"一次, 一旦标记就不再更新。负权边可能让后面的路径更短, 但该节点已经被锁定了。

Dijkstra 负权边反例——分步追踪:

图: $A \rightarrow B$, $A \rightarrow C$, $C \rightarrow D$, $D \rightarrow B$

初始: $\text{dist}=[0(A), \infty(B), \infty(C), \infty(D)]$

第1轮: 选 A, $\text{dist}=[0, 5, -10, \infty]$

第2轮: 选 C(-10最小), $\text{dist}=[0, 5, -10, -9]$

第3轮: 选 D(-9), $\text{dist}=[0, -7, -10, -9]$ $\leftarrow$ B 更新为-7!

第4轮: 选 B(-7) $\leftarrow$ 但 B 的真正最短路是 $A \rightarrow C \rightarrow D \rightarrow B = -7$

$\leftarrow$ 如果 Dijkstra 在第2轮就锁定了 B=5, 错误!

★ Dijkstra 会在第2轮锁定 B=5 (因 $B=5 < D=\infty < C$ 未处理),
 然后 $C \rightarrow D \rightarrow B$ 的 -7 路径永远不会被考虑—B 已锁。

Bellman-Ford (可处理负权): 对所有边做 $V-1$ 轮松弛。第 V 轮还能松弛 $\rightarrow$ 存在负环。

为什么是 $V-1$ 轮? 最短简单路径最多 $V-1$ 条边。第 k 轮松弛后, $\text{dist}[v]$ 保存使用最多 k 条边的最短路径。 $V-1$ 轮后所有可能路径都被考虑。第 V 轮还能更新 $\rightarrow$ 路径长度 $\geq V \rightarrow$ 存在环 $\rightarrow$ 由于每次都松弛 (变短), 此环必为负环。

Floyd-Warshall (多源):

```
for k: for i: for j:
    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

$O(V^3)$, 极其简洁。能检测负环: $\text{dist}[i][i] < 0 \rightarrow$ 有负环。

Floyd 追踪 (4 节点):

初始 dist:

	0	1	2	3
0	[0	3	∞	7]
1	[8	0	2	∞]
2	[5	∞	0	1]
3	[2	∞	∞	0]

k=1 (允许经过节点1):

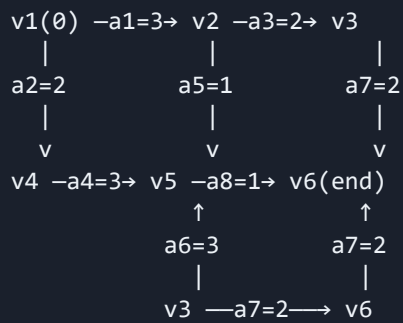
```
dist[0][2] = min( $\infty$ , dist[0][1]+dist[1][2]) = min( $\infty$ , 3+2) = 5 ✓
dist[0][3] = min(7, 3+ $\infty$ ) = 7
dist[2][0] = min(5, 2+8) = 5
dist[3][0] = min(2,  $\infty$ +8) = 2
... 最终 dist[0][2] 从  $\infty$  变为 5
```

11.7 关键路径 (AOE)

这是图论中"图"应用的经典内容，也是项目管理的基础。

问题: 在 AOE (Activity On Edge) 网络中找出决定总工期的关键路径。

AOE 示例网络（6 个事件，8 个活动）：



正向计算最早发生时间 ve ：

```

ve[v1]=0
ve[v2]=ve[v1]+3=3
ve[v3]=max(ve[v2]+2=5, ve[v1]+4=4)=5
ve[v4]=ve[v1]+2=2
ve[v5]=max(ve[v3]+3=8, ve[v4]+3=5, ve[v2]+1=4)=8
ve[v6]=max(ve[v3]+2=7, ve[v5]+1=9)=9
  
```

反向计算最晚发生时间 vl ($v6 \rightarrow v1$)：

```

vl[v6]=9
vl[v5]=vl[v6]-1=8
vl[v3]=min(vl[v6]-2=7, vl[v5]-3=5)=5
...
  
```

关键活动 = $slack=0$ 的活动 ($ve[u]+w=vl[v] \rightarrow$ 在关键路径上)

11.8 图算法选择指南

需求	推荐	复杂度
无权最短路径	BFS	$O(V+E)$
单源最短，非负权	Dijkstra	$O(V^2) / O((V+E)\log V)$
单源最短，含负权	Bellman-Ford	$O(VE)$
多源最短	Floyd	$O(V^3)$
稠密图 MST	Prim $O(V^2)$	$O(V^2)$
稀疏图 MST	Kruskal	$O(E \log E)$
DAG 拓扑序	Kahn	$O(V+E)$
关键路径	AOE	$O(V+E)$

本章小结：

- 图算法是按**问题类型**组织的，而非数据结构——五种问题家族（遍历、拓扑序、MST、最短路、关键路径）对应五个不同方向的拓展

- 工程选择的核心判断依据是图的性质：稀疏/稠密、有向/无向、非负权/带负权——没有普适的最优算法
- DFS 边分类不仅是理论分析工具，更直接指导环检测（回边）、可达性分析（树边）等实用功能
- Dijkstra + 优先队列展示了堆在图算法中的关键应用；Bellman-Ford 的 k 轮松弛思想是所有 DP 型最短路的基础

思考题：

1. 无向图中，证明每条边只能是树边或回边（不可能有前向边或横叉边）。
 2. Dijkstra 用斐波那契堆可以达到 $O(E + V \log V)$ 。解释这两项的来源。简单 $O(V^2)$ 版本在什么场景下反而更优？
 3. 证明：如果所有边权 = 1，Dijkstra 退化为 BFS。
 4. AOE 网中，已知 ve 和 vl 数组，如何找出所有关键活动？如果一条路径上所有活动的松弛时间（slack）都为零，这意味着什么？
-

第六部分：查找

本部分定位：查找结构回答两类根本问题——“某个键在哪里”（关联查找，哈希表）和“某两个元素是否连通”（连通性查找，并查集）。两者的共同主题：**用巧妙的组织形式将查找时间从 $O(n)$ 压到 $O(1)$ 或近似 $O(1)$ 。**

第 12 章 哈希表

难度：★★☆☆☆ | 代码：[hashing.cpp](#)

前置知识：[第 1 章](#) 均摊分析（重哈希的均摊 $O(1)$ ），[第 2 章](#) 链表（分离链接法用链表）。

学习目标：理解哈希函数的映射思想，掌握冲突处理的三种方式及其优劣。

核心思想：通过哈希函数 $h(key)$ 将键直接映射为数组下标 $\rightarrow O(1)$ 平均查找。

12.1 冲突处理

拉链法 (Separate Chaining)

每个槽位存一个链表。冲突就追加。删除直接操作链表。

槽0: $[k1, v] \rightarrow [k3, v]$
槽1: $[k2, v]$
槽2: (空)
槽3: $[k4, v] \rightarrow [k5, v]$

$\alpha = n/m$ (负载因子)
 α 越大 $\rightarrow$ 链表越长 $\rightarrow$ 性能越差

开放地址法 (Open Addressing)

冲突时在表中找下一个空槽。删除需**惰性删除**（标墓碑），否则查找链断裂。

方法	探测序列	问题
线性探测	$h+0, h+1, h+2\dots$	一次聚集——连续占用拉长探测
平方探测	$h+0^2, h+1^2, h+2^2\dots$	二次聚集，表大小需素数 + $\alpha < 0.5$
双重哈希	$h_1 + i \cdot h_2$	最佳，不同 key 不同探测路径

负载因子与探测长度（开放地址法）：

α	成功查找平均探测	不成功查找平均探测
0.5	1.5	2.5
0.75	2.5	8.5
0.9	5.5	50.5

表中可见 $\alpha > 0.75$ 后性能急剧恶化（“膝盖效应”）。这就是为什么哈希表通常在 $\alpha \approx 0.75$ 时扩容。

为什么表大小应取素数？

反例：keys = 6, 12, 18, 24, 30（全是 6 的倍数）
 tableSize = 12: $h(k)=k\%12 \in \{0,6\} \rightarrow$ 只用了 2 个槽！系统聚集
 tableSize = 13: $h(k)=k\%13 \in \{6,12,5,11,4\} \rightarrow$ 5 个槽均匀散布 ✓
 素数表大小让周期性规律失效——这就是数论的功劳。

双重哈希约束： $h_2(key)$ 必须与 tableSize 互质。简单方案：取 tableSize 为素数， $h_2(key)$ 返回 $[1, tableSize-1]$ 范围内的值。

再哈希 (Rehashing)

$\alpha >$ 阈值 (0.7~0.75) $\rightarrow$ 扩容 + 全量重插。单次 $O(n)$ ，均摊 $O(1)$ 。

12.2 完美哈希 (FKS)

代码 [hashing.cpp](#) 中的 `PerfectHash` 类实现了 FKS 双层完美哈希。

思想：第一层哈希分桶，第二层给每个桶一个无冲突的哈希表（表大小 = 桶大小的平方）。如果任何第二层桶还有冲突，换一个新的哈希函数重建。期望总空间 $O(n)$ ，查询严格 $O(1)$ 。

第一层： $h_1(key) \rightarrow$ 桶 b
 第二层： $h_{2_b}(key) \rightarrow$ 桶 b 内的唯一位置
 查询一定 $O(1)$ ——因为第二层保证无冲突！
 代价：构建期望 $O(n)$ ，但最坏可能需要多次重建。

12.3 哈希函数设计

整数： $h(k) = k \% tableSize$ ，tableSize 建议素数（减少规律性冲突）。

字符串（多项式哈希 / Horner 法）：

```
unsigned hash(const string& s) {
    unsigned h = 0;
    for (char c : s) h = h * 31 + c; // 31=素数, h*31=(h<<5)-h
    return h % tableSize;
}
```

12.4 哈希表的局限性

- 无序（无法范围查询）
- 最坏 $O(n)$ （哈希函数差或哈希碰撞攻击）
- 空间利用率低（需要预留空槽）
- 需要好的哈希函数设计

本章小结：

- 哈希以概率换性能——期望 $O(1)$ 的代价是放弃确定性（最坏 $O(n)$ ）和顺序性（无法范围查询）
- 哈希函数是灵魂——好的哈希让一切简单，坏的哈希让一切 $O(n)$
- 两种冲突处理家族各有侧重：拉链法灵活简单，开放地址法缓存友好（连续内存）
- 完美哈希通过双层策略消除了输入分布的依赖，实现严格 $O(1)$ 查询——但构建代价较高
- 理解负载因子 α 是理解哈希表行为的关键——它决定了“满”的程度和性能曲线的转折点

思考题：

1. 平方探测为什么要求 $\alpha < 0.5$ 和素数表大小？不满足这些条件会发生什么？
2. 为 2D 坐标 (x, y) 设计一个哈希函数，使相邻点在哈希表中尽量不聚集。
3. 能否设计一个支持范围查询（如“查找 key 在 100 到 200 之间的所有值”）的哈希表？为什么这从根本上是困难的？

第 13 章 并查集

难度：★★☆☆☆ | 代码：[disjointset.cpp](#)

前置知识：[第 1 章](#) 均摊分析（反阿克曼函数 $\alpha(n)$ 的均摊含义），[第 2 章](#) 链表 / [第 5 章](#) 树（parent 数组模拟树结构）。

学习目标：理解 parent 数组的双重含义设计，掌握路径压缩和按大小合并两个优化。

13.1 核心操作

- **Find(x)**：找 x 所属集合的根
- **Union(x, y)**：合并 x 和 y 的集合

典型应用：Kruskal MST 判环、连通分量统计、动态连通性、社交网络中的“朋友圈”检测。

13.2 parent 数组设计

```
parent[i] < 0 → i 是根, |parent[i]| = 集合大小  
parent[i] ≥ 0 → i 的父节点索引
```

一个数组同时存"父指针"和"集合大小"!

13.3 两大优化

路径压缩: find 时把沿途节点直接挂到根。

```
1→2→3→4(根) → find(1) → 1→4, 2→4, 3→4 (扁平化)  
parent[x] = find(parent[x]); // 核心就这一行!
```

按大小合并 (Union by Size): 小集合挂到大集合下, 控制树高。

```
root1 大小 5, root2 大小 3: root2 挂到 root1  
parent[root1] = -8, parent[root2] = root1
```

按秩合并 (Union by Rank): 用树的高度而非大小作为合并依据, 效果等价。两者在路径压缩配合下均达到 $O(\alpha(n))$ 。

13.4 完整操作示例

初始: [-1, -1, -1, -1, -1, -1, -1, -1]

union(0,1): → parent: [1, -2, -1, -1, -1, -1, -1, -1]

union(2,3): → parent: [1, -2, 3, -2, -1, -1, -1, -1]

union(0,2): find(0)→1, find(2)→3 → 3挂到1

→ parent: [1, -4, 3, 1, -1, -1, -1, -1] (根1大小为4)

find(0): 0→1(根) → 路径压缩:0直接连1

13.5 复杂度

路径压缩 + 按大小合并 → $O(\alpha(n))$, α 是反阿克曼函数。即使 n 等于宇宙原子数, $\alpha(n) \leq 5$ 。

通俗解释: $\alpha(n)$ 的增长速度慢到无法想象—— n 从 0 增长到宇宙原子数, $\alpha(n)$ 只从 0 增长到 5。对所有实际目的, 并查集的每次操作都是 $O(1)$ 。

如果没有路径压缩, 仅靠按秩合并, 并查集退化为 $O(\log n)$ (树高度不超过 $\log n$)。路径压缩是降低到近似常数的关键。

额外应用:

- 图像处理中的连通分量标注
 - Kruskal 最小生成树（见第 11 章）
 - 增量式连通性查询（“在网络中加了一条新光纤后，A 和 B 是否可以通信？”）
-

本章小结：

- 并查集是数据结构的极简主义杰作——一个整数数组 + 两个操作 = 近似 $O(1)$ 的连通性维护
- 设计中的一条核心教训：把“结构性”信息（父指针）和“元信息”（集合大小/秩）编码到同一个数据域（利用符号）
- 路径压缩揭示了“读操作也可以优化结构”的设计哲学——读时稍增开销，换来未来所有操作的加速
- $\alpha(n)$ 是算法理论中罕见的“可证明近乎常数但不是严格常数”的函数——它连接了理论的下界与实际的可用性

思考题：

1. 只使用路径压缩而不使用按大小合并，最坏复杂度是多少？给出一个达到此最坏情况的操作序列。
 2. 给定一个社交网络的用户好友关系列表，设计算法统计独立社交圈的数量。复杂度是多少？
 3. Tarjan 证明 find 操作（路径压缩后）是 $O(\alpha(n))$ 而非 $O(1)$ 。为何不能达到严格 $O(1)$ ？直觉上什么限制了它？
-

第七部分：排序

本部分定位：排序不仅是算法竞赛的高频考点，更是数据库 ORDER BY、搜索引擎相关性排序、推荐系统的底层基础。本部分从稳定性概念出发，逐级递进： $O(n^2)$ 简单排序 $\rightarrow O(n \log n)$ 分治排序 $\rightarrow$ 非比较排序的 $O(n)$ 突破 $\rightarrow$ 外部排序的大数据方案。

第 14 章 排序算法

难度：★★☆☆☆ | 代码：[sort.cpp](#)

前置知识：[第 1 章](#) 复杂度分析（比较排序下界 $\Omega(n \log n)$ ），[第 7 章](#) 堆（堆排序基于堆结构），[第 5 章](#) 二叉树遍历（归并排序的分治递归 = 后序遍历的变体）。

学习目标：掌握稳定性概念，理解 $O(n^2)$ 和 $O(n \log n)$ 排序的原理和适用场景，了解非比较排序的突破原理和外部排序的基本思路。

14.1 基本概念

稳定性：相等元素排序后相对顺序不变 $\rightarrow$ 可以先按姓名排再按年龄排，年龄排序保持同名有序。

比较排序下界：基于比较的排序最优 $\Omega(n \log n)$ 。

证明思路（决策树模型）：

n 个元素有 $n!$ 种可能排序
决策树的叶节点数 $\geq n!$
二叉树有 L 个叶 $\rightarrow$ 高度 $\geq \lceil \log_2 L \rceil$
 $\rightarrow$ 任何比较排序在最坏情况下至少需要 $\lceil \log_2(n!) \rceil$ 次比较
 $\rightarrow \log_2(n!) = \Omega(n \log n)$ （斯特林公式）
 $\rightarrow$ 比较排序下界 = $\Omega(n \log n)$

计数/桶/基数通过**不比较**突破了此下界——它们依赖键的额外性质（如范围有限）。

14.2 插入排序

类似打牌时整理手牌——取新牌，在已排序牌中找位置插入。

```
arr = [5, 2, 4, 6, 1, 3]
```

```
i=1: [5|2,4,6,1,3] → [2,5|4,6,1,3]
```

```
i=2: [2,5|4,6,1,3] → [2,4,5|6,1,3]
```

```
i=3: [2,4,5|6,1,3] → 不动
```

```
i=4: [2,4,5,6|1,3] → [1,2,4,5,6|3]
```

```
i=5: [1,2,4,5,6|3] → [1,2,3,4,5,6]
```

- 最好 $O(n)$, 最坏 $O(n^2)$, 稳定
- $n < 50$ 时比复杂排序更快 (常数小, TimSort 用于小数组)

14.3 $O(n \log n)$ 三大排序

归并排序: 分治, $O(n \log n)$, 稳定, 需 $O(n)$ 空间。适合外排序。

快速排序: 分区(partition) + 递归, 平均 $O(n \log n)$, 不稳定, 常数因子最小。

Lomuto 分区追踪 (pivot = arr

hi

= 6):

```
arr = [3, 8, 2, 5, 1, 4, 7, 6], pivot=6
i=-1, j=0..6:
  j=0: 3≤6 → i=0, swap(0,0): [3,8,2,5,1,4,7,6]
  j=1: 8>6 → skip
  j=2: 2≤6 → i=1, swap: [3,2,8,5,1,4,7,6]
  j=3: 5≤6 → i=2, swap: [3,2,5,8,1,4,7,6]
  j=4: 1≤6 → i=3, swap: [3,2,5,1,8,4,7,6]
  j=5: 4≤6 → i=4, swap: [3,2,5,1,4,8,7,6]
  j=6: 7>6 → skip
最后: swap(i+1=5, pivot=7): [3,2,5,1,4,6,7,8]
pivot 6 在正确位置(index=5) ✓
```

- Lomuto 分区 (简单): pivot=arr

hi

, 维持左 $\leq$ pivot < 右

- Hoare 分区 (高效): 双指针向中间扫描
- 最坏 $O(n^2)$ → 随机 pivot 或三数取中避免

堆排序: 建堆 $O(n)$ + 依次弹出 $O(n \log n)$, $O(1)$ 空间, 不稳定。

14.4 逆序对计数

$i < j$ 且 $arr[i] > arr[j]$ 的数对 (i,j) 。

归并过程中: $arr[i] > arr[j] \rightarrow arr$

$i \dots mid$

都 $> arr$

j

→ 增加 `mid-i+1` 个逆序对。 $O(n \log n)$ 。

14.5 Shell 排序（希尔排序）

插入排序的改进：对间隔 gap 的子序列分别插入排序，逐步缩小 gap 。

增量序列：

- Shell 原始： $n/2, n/4, \dots, 1 \rightarrow O(n^2)$ 最坏
- Hibbard： $1, 3, 7, 15, \dots, 2^k-1 \rightarrow O(n^{3/2})$
- Sedgewick： $\rightarrow O(n^{4/3})$ 实践中最快

14.6 TimSort

[sort.cpp](#) 中实现了完整的 TimSort。

Python 和 Java 的内置排序均使用 TimSort——一种自适应于“部分有序”数据的混合排序，融合了归并排序的分治和插入排序的小数据优势。核心思想：识别并利用输入中的有序片段（run），减少不必要的比较和移动。

14.7 非比较排序

计数排序 $O(n+k)$ ：统计每个值的频次 → 展开。要求 k 不能太大。**稳定实现**需要用累加频率来确定每个元素的位置。

基数排序 $O(d \cdot n)$ ：从低位到高位，每位做稳定排序。**必须从低位开始**——高位排序“优先级”高，低位先排保证高位相同时低位有序。

14.8 外部排序

当数据量超过内存容量时，排序必须利用磁盘。外部排序是排序理论中的重要内容。

两阶段流程：

阶段 1：分块内部排序
将大数据文件分成 M 块（每块能装入内存）
每块装入内存 $\rightarrow$ 快速排序 $\rightarrow$ 写回磁盘作为有序块（run）
复杂度： $O(N \log M)$

阶段 2：K 路归并
使用最小堆从 K 个有序块中选出最小的记录
每次堆 pop 后从对应的块补充新记录
输出到最终文件
复杂度： $O(N \log K)$

总 I/O: $O(N/B \times (1 + \log_K (N/M)))$ 其中 B 是磁盘页大小

败者树优化：K 路归并中，每次选最小需 $O(\log K)$ 比较。败者树将"重赛"次数减半——输者存树中，胜者向上比。堆需要两次比较（与左右子），败者树只需一次。

实际引用：

- MySQL ORDER BY 在数据超出 `sort_buffer_size` 时使用外部归并排序
- PostgreSQL 的外部排序使用 K 路归并 + 败者树
- Hadoop/Spark 的 Shuffle 阶段本质是外部排序

14.9 排序选择指南

场景	推荐	理由
小数据 ($n < 50$)	插入	常数小
通用	快速	平均最快
需稳定	归并	稳定 + $O(n \log n)$
内存极有限	堆	$O(1)$ 空间
整数范围小	计数	突破 $O(n \log n)$
磁盘排序	外部归并	k 路归并
几乎有序	插入	接近 $O(n)$
通用库实现	TimSort	自适应

14.10 十大排序速览

算法	平均	最坏	空间	稳定
插入	$O(n^2)$	$O(n^2)$	$O(1)$	✓
冒泡	$O(n^2)$	$O(n^2)$	$O(1)$	✓
选择	$O(n^2)$	$O(n^2)$	$O(1)$	X
希尔	$O(n^{1.3})$	$O(n^2)$	$O(1)$	X
归并	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
快速	$O(n \log n)$	$O(n^2)$	$O(\log n)$	X
堆	$O(n \log n)$	$O(n \log n)$	$O(1)$	X
计数	$O(n+k)$	$O(n+k)$	$O(k)$	✓
桶	$O(n+k)$	$O(n^2)$	$O(n+k)$	✓
基数	$O(d \cdot n)$	$O(d \cdot n)$	$O(n)$	✓

本章小结：

- 排序算法有三个层次的设计智慧：(1) 利用数据特性（插入排序对"几乎有序"数据接近 $O(n)$ ），(2) 分而治之（归并/快速），(3) 彻底超越比较模型（计数/基数/桶）
- 稳定性不是闲谈——它是多键排序的基础（先按成绩排再按学号排）
- 比较排序的 $\Omega(n \log n)$ 下界既是理论瑰宝也是实践指南——一旦需要 $n \log n$ 次操作，对于大数据就要考虑外排序
- 外部排序展示了"内存-磁盘"两级存储下算法设计的根本范式转变——I/O 次数而非 CPU 时间成为真正的瓶颈
- TimSort 代表了工业级排序的务实哲学——"最佳"算法不是数学上最优的，而是对真实数据模式最优的

思考题：

1. 用决策树模型证明 $\Omega(n \log n)$ 下界。
2. 给定 10,000 个范围在

$0, 100$

的整数，选哪种排序最好？为什么？
3. 设计一个算法对 1TB 文件进行排序，机器有 1GB RAM。估算 I/O 操作次数。
4. 为什么快速排序在实践中的平均速度优于归并排序，尽管两者有相同的渐近复杂度？（从常数因子、缓存、分支预测等方面思考）。
5. 外部排序的 K 路归并中，K 值增大对总 I/O 次数有什么影响？K 太大有什么缺点？

第八部分：高级字符串

本部分定位：第 4 章 KMP 解决了“在文本中找一个模式”的问题。高级字符串结构处理更根本的挑战——如何高效回答**“关于一个文本的所有子串的所有问题”**？后缀树和后缀数组把文本的所有后缀组织成可搜索的结构，将字符串问题转化为树/数组上的查询问题。这是一种极致的“预处理一次，查询多次”范式。

第 15 章 后缀树与后缀数组

难度：★★★★★ | 代码：[suffixtree.cpp](#) / [suffixarray.cpp](#)

前置知识：[第 4 章 KMP](#)（对“预处理使快速查询成为可能”有体会），[第 5 章](#) 二叉树遍历（后缀树是压缩 Trie），[第 14 章](#) 排序（后缀数组构建需要排序）。

学习目标：理解“子串=后缀的前缀”这一核心洞察，了解后缀树的压缩原理和 Ukkonen 算法思路，掌握后缀数组的结构与应用。

15.1 核心洞察

任意子串都是某个后缀的前缀。

$S = \text{"banana"}$ ：子串 "nan" = 后缀 $\text{"nana"} (i=2)$ 的前 3 个字符。

→ 高效组织所有后缀 = 高效解决所有子串问题。

15.2 后缀树

后缀 Trie → $O(n^2)$ 节点（不可接受）→ 后缀树（压缩链为边）→ $O(n)$ 节点！

压缩前后对比 ($S = \text{"banana\$"})$ ：

```

      root
      /  |  |  \
      b  a  n  $
      /  /  \  \
      a  n  $  n
      /      |      \
      n      a      a
      |      |      |
      a      n      n
      |      |      |
      n      a      a
      |      |      |
      a      $      $
      |
      $
      |
      $

```

```

      root
      /  |  \
     a   ba  na  $
    / \   |   |
   $ na na$ na$
      |
     na$
      |
      $

```

1. 根→叶路径 = 一个后缀
2. 内部节点 ≥ 2 子节点
3. 最多 $2n$ 个节点 $\rightarrow O(n)$ 空间
4. 模式匹配 $O(m)$ ，与文本长度无关

$O(n)$ 在线构建。三个核心概念：

- **活动点 (Active Point)**: 记录上次插入结束的位置, 避免每次从根重新开始。处理新字符时, 从活动点继续而非从根重来
- **剩余后缀数 (Remainder)**: 当前还有几个后缀需要显式插入。利用后缀链接来减少实际插入次数
- **后缀链接 (Suffix Link)**: 内部节点间的快捷指针。如果某内部节点代表子串 “ $x\alpha$ ”, 则其后缀链接指向代表子串 “ α ” 的节点——这使得从上一个后缀的插入位置跳到下一个后缀的插入位置只需 $O(1)$

 $0 \dots i$

构建了隐式后缀树。活动点记住"最后插入到哪了", 避免冗余回溯。后缀链接像地铁换乘通道——让你从一条路径快速跳到另一条路径而不必回到起点。

后缀树的紧凑替代。将所有后缀按字典序排序，存起始索引。

```
S = "banana"
后缀排序: a(5) < ana(3) < anana(1) < banana(0) < na(4) < nana(2)
SA = [5, 3, 1, 0, 4, 2]    (后缀起始索引)
rank = SA 的逆映射        (rank[i] = 后缀 i 的排序位置)
```

height



的 LCP（最长公共前缀）长度。

LCP 数组应用表：

任务	利用 SA + LCP 的解法	复杂度
模式匹配	SA 上二分查找	$O(m \log n)$
最长重复子串	$\max(\text{height}[i])$	$O(n)$
最长公共子串（两串）	拼接 + “#” + 找 max height 且来自不同串	$O(n)$
不同子串个数	$n(n+1)/2 - \sum \text{height}[i]$	$O(n)$

15.5 后缀树 vs 后缀数组

维度	后缀树	后缀数组
空间	$O(n)$ ，常数大（边、子串范围）	$O(n)$ ，紧凑（一个 int 数组）
构建	$O(n)$ Ukkonen（实现复杂）	$O(n \log n)$ 倍增法（简洁） / $O(n)$ DC3/SA-IS
模式匹配	$O(m)$ 直接沿树走	$O(m \log n)$ 二分 / $O(m + \log n)$ 配合 LCP
实现难度	极高	中等
应用场景	理论研究、生物信息学	竞赛编程、生物信息学（BWA）

实践中，后缀数组因其实现简洁和空间紧凑而更通用。竞赛编程中几乎全部使用后缀数组 + LCP，而非后缀树。

本章小结：

- "子串 = 后缀的前缀"是字符串数据结构的核心公理——它把一个看似 $O(n^2)$ 子串"的问题转化为 $O(n)$ 后缀"的问题
- 后缀树通过边压缩将 $O(n^2)$ 节点的 Trie 压缩到 $O(n)$ ，而 Ukkonen 算法在 $O(n)$ 时间内在线完成这一构建——是算法设计中的罕见杰作
- 后缀数组比后缀树更实用的关键在于：它只保存起始索引，将结构信息"外置"到 LCP 数组中，从而在简单性和功能之间取得优秀平衡
- 这两种结构都是"预处理一次，回答多次"范式的极致体现—— $O(n)$ 预处理后，几乎任何子串问题都能在 $O(m)$ 或 $O(m \log n)$ 内回答

思考题：

1. 给定 `S = "abracadabra"` 的后缀数组和 LCP 数组，找出最长重复子串及其出现位置。
2. 一个长度为 n 的字符串的后缀树最多有多少个内部节点？证明你的答案。
3. Kasai 算法在 $O(n)$ 时间内从后缀数组计算 LCP 数组。关键不变量是：对于原始字符串中连续的后缀 rank

0

, rank

1

, ..., `h = max(0, h-1)`。解释这个不变量为什么成立。

附录

附录 A：数据结构选择速查

需求	推荐结构	复杂度
快速增删，不需频繁查找	链表	$O(1)$ 插入/删除
快速查找 + 有序遍历	平衡 BST	$O(\log n)$
最快查找（无序）	哈希表	$O(1)$ 平均
优先级队列	堆	$O(\log n)$ push/pop
局部性访问	伸展树	均摊 $O(\log n)$
大量数据存磁盘	B 树 / B+ 树	$O(\log_m n)$
判断连通性	并查集	$\approx O(1)$
多维搜索	KD 树	平均 $O(\log n)$
模式匹配（单次）	KMP	$O(n+m)$
模式匹配（多次）	后缀树 / 后缀数组	$O(m)$ / $O(m \log n)$
范围查询（数据库）	B+ 树	$O(\log_m n + k)$
Top-K 问题	最小堆	$O(n \log k)$
数据流中位数	双堆	插入 $O(\log n)$
滑动窗口最大值	单调队列	$O(n)$
区间最值查询	稀疏表 / 线段树	$O(1)$ / $O(\log n)$
大文件排序	外部归并	$O(N \log N)$ I/O

附录 B：常见面试题与练习

线性结构

1. 反转链表（迭代 + 递归）
2. 检测链表是否有环 → 找环入口
3. 合并两个有序链表 → 合并 K 个有序链表
4. 删除链表倒数第 N 个节点

5. 用两个栈实现队列 / 用两个队列实现栈
6. 最小栈 ($O(1)$ 获取最小值)
7. 有效的括号匹配
8. 单调栈 — 下一个更大元素 / 每日温度 / 柱状图中最大矩形
9. 循环队列实现

树

1. 二叉树的最大深度 / 最小深度
2. 判断是否为平衡二叉树
3. 验证 BST 合法性
4. 二叉树的最近公共祖先 (LCA)
5. 二叉树的层序遍历 / 锯齿形遍历
6. 路径总和 (是否存在 + 所有路径)
7. AVL 树手动构建 (给定插入序列, 画图)

堆与排序

1. 数组中的第 K 大元素 (Top-K)
2. 数据流中的中位数 (双堆)
3. 归并排序 / 快速排序手写实现
4. 求逆序对数量
5. 外部 K 路归并设计
6. 手写堆排序 (建堆 + 依次弹出)
7. 快速选择 (QuickSelect)

哈希与查找

1. 两数之和 (Two Sum)
2. 最长连续序列
3. 字母异位词分组
4. LRU 缓存 (哈希表 + 双向链表)

图

1. 课程表 (拓扑排序判环)
2. 岛屿数量 (DFS/BFS 连通分量)
3. 网络延迟时间 (Dijkstra)
4. 最小生成树 (Kruskal / Prim)
5. 关键路径上的活动识别

字符串

1. 实现 `strStr()` (KMP)
2. 最长回文子串 (中心扩展 / Manacher)
3. 最长公共前缀
4. 无重复字符的最长子串 (滑动窗口)

附录 C：常见陷阱与注意事项

指针陷阱

1. `delete` 后未置 `nullptr` → 野指针，后续使用崩溃
2. `new` 后忘记 `delete` → 内存泄漏
3. 默认拷贝构造只复制指针 → 两个对象共享一棵树 → `double free`
→ 自定义深拷贝 `or = delete`
4. 递归删除树 → 必须后序遍历 (先子后父)

递归陷阱

1. 忘记 `base case` → 无限递归爆栈
2. 递归返回值不接收 → `BST` 删除/插入时丢失父子链接
3. 深度过大 → 栈溢出 (改用迭代 `or` 尾递归)
4. 重复计算 → 使用备忘录 (记忆化搜索)

边界陷阱

1. 取 $(i-1)/2$ 前确保 $i > 0$
2. `pop()/top()` 前先检查 `!IsEmpty()`
3. 循环队列: $(rear+1)\%cap == front$ 判满 (牺牲一个位置)
4. `int` vs `size_t`: 有符号和无符号混用导致意外行为
5. 链表操作时注意处理空链表和单节点链表的特殊情况

算法选择陷阱

1. 稠密图上用 `Kruskal` 不用 `Prim` $O(V^2)$ → 忽略 `Prim` 在稠密图上的优势
2. 需要范围查询时用哈希表 → 哈希表不支持有序遍历
3. 小数据 ($n < 50$) 上用快速排序 → 插入排序常数更小
4. 数据已排好序时 `BST` 逐个插入 → 退化为链表 $O(n^2)$
5. 对流式数据用归并排序 → 归并排序需要看到全部数据

性能陷阱

1. 哈希表最坏 $O(n)$: 选择好的哈希函数 + 控制负载因子
2. BST 退化成链表: 使用自平衡变体 (AVL/红黑树)
3. 递归建树 $O(n^2)$: 每次取中位数确保 $O(n \log n)$
4. 大对象传值: 传引用避免 $O(n)$ 拷贝

本文档基于 18 个可编译运行的 C++ 文件编写，按经典教材体系编排，从绪论到高级字符串，递进讲解。每个章节配备前置知识、内容详解、本章小结和思考题，适合自学和教学参考。

学习建议：从头到尾按顺序阅读，每学完一章尝试回答思考题。代码实现细节请参阅各章标注的 .cpp 源文件。